Atmospheric Starship Controller

Chapter 1: Getting Started

1.1 Introduction

Welcome to the Veridian Starship Controller asset package! This documentation will guide you through the setup, configuration, and advanced features of this comprehensive toolkit, enabling you to create compelling atmospheric flight and combat experiences in your Unity projects.

1.1.1 Overview

The Veridian Starship Controller is a robust, modular toolkit designed for creating physics-based atmospheric starship flight, combat, and racing mechanics. It provides a complete framework that balances realistic flight dynamics with arcade responsiveness, suitable for a wide range of game genres.

Whether you are developing high-speed racing, intense dogfighting simulations, or complex AI squadron battles, this asset provides the core systems needed to bring your vision to life.

Key Features:

- **Realistic Physics Flight Model:** A sophisticated atmospheric flight controller (`AtmosphericStarshipController`) that handles thrust, rotation, customizable aerodynamic drag, ground effects, and G-force limitations.
- **Configurable AI Pilot System:** A flexible AI framework featuring the `AiBrain` for strategic decisions (using prioritized actions and conditions) and the `SimpleAiPilot` for flight execution (using PID controllers for smooth navigation).
- **Modular AI Behaviors:** Easily extendable behavior system with pre-built behaviors like Attack, Sentry, Patrol, Gofer, and Loiter, all configured via ScriptableObjects.
- **Combat and Weapon Systems:** Supports primary (lasers), secondary (guided or unguided rockets), and bombardment weapons with configurable stats, gimbaling, and projectile handling.
- **Player Controller & Camera Setup:** Includes a ready-to-use player driver (`PlayerShipDriver`) utilizing Unity's new Input System, and a dynamic third-person camera (`StarshipCameraController`) featuring smoothing, collision detection, acceleration lag, and G-force effects.
- **Faction and Lifecycle Management:** A centralized `FactionManager` handles team rosters, roles (Leader/Follower), respawning, and provides tactical intelligence for the AI systems.
- **ScriptableObject Configuration:** Tune ship properties, AI personalities, weapon stats, and environmental settings easily using dedicated ScriptableObject profiles.
- **Performance Optimization (LERP Mode):** Switch distant AI agents to a low-cost kinematic "LERP" (Linear Interpolation) mode to maintain performance in large-scale scenarios.

1.1.2 Package Contents

Upon importing the asset, you will find the following main folder structure within `Assets/Veridian/Starship/`:

- **AI:** Contains all scripts related to the AI system, including the Brain, Pilot, behaviors, conditions, and configuration assets (Personalities, Actions).
- **Core:** The foundational scripts, including the main flight controller (`AtmosphericStarshipController`), `ShipProperties`, `HealthComponent`, `FactionManager`, `SensorySystem`, and related interfaces.
- **Demo:** Includes example scenes, models, and assets showcasing the asset's capabilities. This folder can be safely removed in a production build.
- **Documentation:** Contains this documentation PDF.
- **Player:** Scripts specific to the player experience, such as the `PlayerShipDriver`, `InputProvider`, and `StarshipCameraController`.
- **Prefabs:** Ready-to-use GameObjects, including example Player and AI ships, utility prefabs, and projectiles.
- **Profiles (ScriptableObjects):** Contains configuration assets for ship flight characteristics, AI personalities, weapon loadouts, and environmental settings.
- **Weapons:** Scripts and assets related to the weapon systems, including the `ShipWeaponController` and projectile logic.

1.1.3 Intended Use Cases

This asset is well-suited for a variety of game types, including:

- Arcade space combat games (e.g., "Rogue Squadron" or "Ace Combat" style gameplay).
- Atmospheric flight simulators with mission objectives.
- Starship racing mechanics requiring precise physics and competitive AI.
- Games featuring AI squadron battles and cooperative AI teammates.
- Open-world games featuring pilotable vehicles in atmospheric environments.

1.4 Core Design Philosophy

The asset is designed with three core principles:

1. **Modularity:** Systems are decoupled. The core flight controller relies on the `IShipDriver` interface, allowing seamless switching between human (`PlayerShipDriver`) and AI (`SimpleAiPilot`) control. AI behaviors are implemented as distinct classes (`ISimpleAiBehavior`), making the system easily extensible.
2. **Configurability:** We heavily leverage Unity's ScriptableObjects. This allows designers to tune and create variations of ships, AI behaviors, and weapons without writing code, facilitating rapid iteration.
3. **Scalable Fidelity:** The dual-mode (Physics/LERP) approach of the `AtmosphericStarshipController` allows developers to balance fidelity and performance. High-fidelity physics is used for the player and nearby AI, while distant AI can switch to a low-cost LERP mode.

1.5 Support Information

We are committed to supporting this asset. If you encounter issues, have questions, or require assistance:

- **Online Documentation:** [Link to Online Documentation]
- **Unity Forum Thread:** [Link to Forum Thread]
- **Support Email:** [Support Email Address]

1.2 Quick Start

This chapter provides the fastest way to get a functional starship flying in a new scene using the provided prefabs.

1.2.1 Prerequisites & Installation

Before installing, please ensure your project meets the following requirements:

- **Unity Version:** Unity 2025.x (Unity 6) or later.
- **Render Pipeline:** This asset is designed for and requires the **Universal Render Pipeline (URP)**. Ensure URP is installed and configured in your project settings.
- **Input System:** This asset utilizes Unity's new **Input System** package. Ensure it is installed via the Package Manager and enabled in your Project Settings (`Edit` > `Project Settings` > `Input System Package`).

Installation:

1. Open your Unity project.
2. Go to `Window` > `Package Manager`.
3. Locate the Veridian Starship Controller asset (via "My Assets" or by importing the local package).
4. Click `Import` and ensure all files are selected in the import dialogue.

1.2.2 Running the Demo Scene

We highly recommend exploring the demo scene first to see the asset's features in action and understand how the components are configured. This is the best place to see exactly how the core manager scripts should be attached to GameObjects in a production environment.

1. In the Project window, navigate to: `Assets/Veridian/Starship/Demo/Scenes/`.
2. Double-click `MainDemo.unity` to open the scene.
3. Press the Play button in the Unity Editor.

The demo showcases player flight controls, AI behaviors (patrolling, dogfighting), weapon systems, and the dynamic camera system in a fully functional, ready-to-play environment.

A Note on Guided Missiles: While the demo scene provides a great overview of flight and standard combat, the ships in the demo are currently equipped with unguided secondary rockets by default. However, the asset fully supports guided "fire-and-forget" missiles! To test this out yourself:

1. Open the `WeaponStats` ScriptableObject for the secondary weapon used in the demo.
2. Check the `isGuided` boolean.
3. Ensure the weapon prefab assigned is using the `GuidedRocketProjectile` script instead of the standard `RocketProjectile` script.
4. In-game, hold the Aim Mode button (Right Mouse Button) to lock onto a target before firing your secondary weapon.

A Note on AI Behaviors: The AI ships in the demo scene are configured to showcase a specific subset of the available behaviors (primarily patrolling and attacking). However, the Veridian AI framework includes many more built-in tactical behaviors, such as Loiter, Flee, Takeoff, Landing, and Gofer. Because the AI is built on a modular Utility AI system, you can easily add, remove, or re-prioritize these behaviors by modifying the `AiActionSO` lists on the ship's `AiBrain` component in the Inspector.

1.2.3 Basic Scene Setup (Step-by-Step)

Follow these steps to set up a new, empty scene with a player-controlled starship.

1. Create a New Scene: Go to File > New Scene. Select the URP Basic 3D template if available.

2. Add Basic Ground: Right-click in the Hierarchy and select 3D Object > Plane. Set the Plane's scale to (100, 1, 100) to create a large ground area. Ensure the ground object is on a physics layer that the ship's sensors can detect (e.g., "Default").

3. Add Essential Manager Scripts: Rather than using rigid prefabs, the Veridian core systems are modular scripts. Right-click in the Hierarchy and select **Create Empty**. Name this new GameObject "Managers". In your Project window, search for the following scripts and drag them onto your "Managers" GameObject:

- **EnvironmentManager:** Required for the physics model. It sets the global atmospheric drag properties. *(Note: You will need to assign an EnvironmentProfileSO to this component in the inspector).*
- **FactionManager:** Required for ship registration, lifecycle management (respawning), and targeting systems.
- **GameInitializer:** This script automatically finds the player ship, the main camera, and the input provider, linking them together seamlessly at runtime.

*(Note: You do not need to manually add the **InputProvider**. The GameInitializer is designed to automatically generate this component at runtime if it is not found in the scene!)*

4. Add the Player Ship: Navigate to `Assets/Veridian/Starship/Prefabs/Player/` (or `Prefabs/Ships/`). Drag a `Player_Starship_Prefab` variant into the scene. Position the ship slightly above the ground, e.g., (0, 10, 0). *Why:* This is your controllable ship, preconfigured with the AtmosphericStarshipController, PlayerShipDriver, HealthComponent, and ShipWeaponController.

5. Configure the Camera: Ensure the "Main Camera" object (created by default in a new scene) exists in the Hierarchy and is tagged as `MainCamera`. The GameInitializer will automatically add the StarshipCameraController component to this camera at runtime and link it to the player ship.

6. Press Play: Click the Play button. *Expected Result:* The camera should snap behind the ship, the cursor should be locked and hidden (managed by the auto-generated GameStateManager), and you should have full control of the starship.

Default Controls

- **W/S:** Forward/Reverse Thrust
- **A/D:** Yaw Left/Right
- **Q/E:** Roll Left/Right
- **Mouse Movement:** Pitch and Yaw
- **Space/Ctrl:** Vertical Thrust Up/Down
- **L-Shift:** Boost
- **LMB (Left Mouse Button):** Fire Primary (Lasers)
- **RMB (Right Mouse Button):** Hold to Engage Target (Auto-aim and Missile Lock)
- **F:** Fire Secondary (Rockets)
- **G:** Drop Bomb
- **Esc:** Pause

1.2.4 Key Components & Prefabs Overview

Here is a brief overview of the essential prefabs and scripts used in the Quick Start:

- **Player_Starship_Prefab (Prefab):** A complete, ready-to-fly player ship. It includes all necessary components: physics controller, player driver, weapons, health, and identity.
- **EnvironmentManager (Script):** A singleton manager required for the physics flight model. It provides the aerodynamic drag settings based on the active EnvironmentProfileSO.
- **FactionManager (Script):** A singleton manager required for targeting, AI, and respawning. It tracks all ships in the scene.
- **GameInitializer (Script):** A utility component that simplifies scene setup by automatically linking the player, camera, and input systems.
- **InputProvider (Auto-Generated):** A singleton manager that handles player input mapping. Created automatically by the GameInitializer.
- **GameStateManager (Auto-Generated):** A singleton manager that controls the pause state and cursor management. Created automatically when needed.

Chapter 2: Core Starship Systems

This chapter details the fundamental components that form the architecture of the Starship Controller asset. Understanding these core systems is essential for configuring, utilizing, and extending the asset effectively. These scripts handle everything from the physics simulation and movement logic to the ship's identity, health, weapons, and sensory input.

The architecture is designed to be modular, utilizing interfaces and ScriptableObjects to decouple logic from data and allow for flexibility in implementation.

2.1 Core Concepts and Interfaces

Before diving into the specific components, it's essential to understand the key interfaces and data structures that facilitate communication between different parts of the system.

2.1.1 The IShipDriver Interface

The `IShipDriver` interface (`IShipDriver.cs`) is a critical abstraction layer that decouples the *control* of a starship from its *movement*. It defines the contract for any entity—whether a human player or an AI routine—that wishes to pilot a starship.

Purpose: To provide a standardized way for the `AtmosphericStarshipController` to receive input without needing to know *who* is providing it. This allows easy swapping between player control and AI control on the same starship prefab.

Key Methods:

- `void AssignController(AtmosphericStarshipController controller)`: Called when the driver is linked to a specific ship controller. The driver should cache this reference.
- `void ReleaseController()`: Called when the driver is unlinked.
- `ShipInputState GetDesiredInputState()`: The core method, called every frame by the controller. The driver must return a `ShipInputState` struct containing the desired inputs for that frame.
- `bool IsActivelyControlling()`: Indicates if the driver is currently providing meaningful rotational input.

Implementations: The asset includes two primary implementations: `PlayerShipDriver` (for human input) and `SimpleAiPilot` (for AI behaviors). While their internal logic is covered in later chapters ("Player Setup" and "AI System"), both communicate with the `AtmosphericStarshipController` exclusively through this interface.

2.1.2 ShipInputState Struct

The `ShipInputState` struct (`ShipInputState.cs`) is the data packet sent from the `IShipDriver` to the `AtmosphericStarshipController` every frame.

Fields:

- **Axis Inputs (-1.0 to 1.0):** `Thrust` (Forward/Reverse), `Vertical` (Up/Down), `Roll`.
- **Rotation Inputs (Scaled values):** `Pitch`, `Yaw`. These values can exceed the -1 to 1 range, especially when driven by mouse input scaled by sensitivity, allowing the controller to interpret the desired rate of rotation.
- **Boolean States:** `Boost`, `FirePrimary`, `FireSecondary`, `FireBomb`.
- **Targeting Data:**
    - `Vector3 AimPosition`: The world-space coordinate the driver is aiming at. Used for unguided weapons and gimbaling.
    - `bool IsTargetEngaged`: A crucial flag indicating if the driver has a lock on a target. This informs the `ShipWeaponController` whether to fire secondary weapons as guided missiles (if applicable).

2.2 Physics and Movement

The heart of the asset is the physics controller, responsible for interpreting input and translating it into realistic (or stylized) atmospheric flight.

2.2.1 AtmosphericStarshipController.cs

The `AtmosphericStarshipController` is the primary component managing a starship's movement, physics interactions, and state management. It acts as the central hub, connecting the `IShipDriver`, `ShipProperties`, `ShipSensorySystem`, and `ShipWeaponController`.

Design Philosophy: It implements a custom, arcade-style atmospheric flight model using Unity's Rigidbody physics. It emphasizes responsive control and "feel" over strict aerodynamic simulation, utilizing custom drag models, ground effects, and G-force limiting. It also includes a performance optimization mode (LERP) for distant AI.

Configuration and Setup: This component requires a `Rigidbody`, `ShipProperties`, `HealthComponent`, and `ShipWeaponController` on the same GameObject.

- `isAI` (bool): A flag typically set by the driver. This influences `Rigidbody` settings, such as enabling `RigidbodyInterpolation.Interpolate` only for the player ship (for smooth camera movement), while AI ships use `None` for performance.
- `defaultDriverComponent` (MonoBehaviour): Assign the default `IShipDriver` (e.g., `SimpleAiPilot` or `PlayerShipDriver`) here.

The Update Loop (Input and State): The controller gathers input in `Update()` by calling `GetDesiredInputState()` on the active driver. It also manages non-physics systems like boost fuel regeneration (`HandleBoostLogic`), thrust ramping (`HandleRamping`), and weapon firing (`HandleWeapons`) in `Update()`.

The FixedUpdate Loop (Physics): All physics calculations occur in `FixedUpdate()`:

1. `CalculateGForce()` and `CalculateGForceLimiter()`
2. `ApplyThrust()` (Forward, reverse, vertical)
3. `ApplyGroundEffect()` (Requires `ShipSensorySystem`)
4. `ApplyRotation()` (Torque)
5. `ApplyAerodynamicDrag()` (Requires `EnvironmentManager`)
6. `ClampVelocity()`

2.2.1.1 Force Application Details

Forces are applied using `ForceMode.Force`.

- **Thrust:** Applied along the local Z (forward/reverse) and Y (vertical) axes. Power is scaled by the current input level, boost multiplier, and G-force governor.
- **Rotation (Torque):** Applied as relative torque. Rotational power is modified by the `maneuverabilityCurve` (based on current speed) and the G-force governor.

Aerodynamic Drag Model: The controller implements a custom aerodynamic drag model, overriding Unity's built-in `linearDamping`.

- **Model:** Combines Linear Drag (dominant at low speeds) and Quadratic Drag (dominant at high speeds, proportional to velocity squared).

- **Directional Drag:** Drag is calculated separately for forward and sideways velocity. Sideways drag is typically configured to be much higher, encouraging forward flight and resisting drifting.
- **Environment Dependency:** The base drag coefficients are defined in the `EnvironmentProfileSO` (see Section 3.4.2).

Ground Effect: When the `ShipSensorySystem` detects the ground within the configured `groundEffectMaxAltitude`, an upward force (`Vector3.up`) is applied. The strength is determined by the `groundEffectCurve` in `ShipProperties`, providing a "cushion" of air near the surface.

2.2.1.2 Boost Mechanics

The boost system provides temporary increases in acceleration and top speed.

- **Fuel:** Managed by `currentBoost` and `maxBoost`.
- **Drain/Recharge:** Fuel drains when boosting and recharges after a configurable delay.
- **Effect:** The `CurrentBoostMultiplier` is applied to both forward thrust power and the `maxSpeed` cap.
- **Boost Speed Modifier Curve:** The `boostSpeedModifierCurve` allows fine-tuning the effectiveness of the boost based on the ship's current normalized speed.

2.2.1.3 G-Force Limiting

To prevent unrealistically sharp maneuvers, the G-Force Limiter can be enabled.

- **Calculation:** `CurrentGForce` is calculated by measuring acceleration (change in velocity over time) and dividing by 9.81 m/s².
- **Limiting:** If `CurrentGForce` exceeds the configured `maxOverallGForce`, a `currentGForceGovernor` (a multiplier between 0.05 and 1.0) is calculated using an inverse function for a smooth limit.
- **Application:** This governor is applied to reduce the effectiveness of all thrust and rotational torques, smoothly reducing responsiveness when the G-force limit is hit.

2.2.1.4 Movement Modes: Physics vs. LERP

The controller supports two movement modes, managed via the public `SetMovementMode()` method:

- `StarshipMovementMode.Physics`: The default, high-fidelity mode. Uses the full `FixedUpdate` loop and the Unity physics engine. Provides realistic interactions and collisions but has a higher CPU cost.
- `StarshipMovementMode.Lerp`: A low-fidelity, kinematic mode designed for performance optimization, typically used for distant AI ships.
    - The `Rigidbody` is set to `isKinematic = true`. Movement is handled in `Update()` by directly manipulating the Transform.
    - Physics calculations (drag, G-force, ground effect) are bypassed.
    - `ShipSensorySystem` is disabled, replaced by simplified altitude checks.

When switching modes, the controller attempts to preserve velocity (stored in `_lerpVelocity`) to ensure a smooth transition between the physics simulation and the LERP approximation.

2.2.2 SecondaryFlightMotion.cs

This component provides visual enhancements to the flight model without affecting the underlying physics.

Purpose: To create a more dynamic visual appearance by automatically banking the ship's model during turns (auto-banking).

Functionality:

- It reads the `Rigidbody.angularVelocity` (specifically the yaw rate).
- It calculates a target bank angle based on the `turnRateSensitivity` and `maxBankAngle`.
- It smoothly rotates the `visualModel` Transform (a child object containing the mesh renderer) on its local Z-axis towards the target angle.

Manual Roll Override: If the driver performs a manual roll, auto-banking is temporarily disabled (for `manualRollCooldown` seconds) to prioritize the pilot's input visually.

Note: This component is purely visual and is automatically disabled if the controller enters LERP mode.

2.3 Configuration and Properties

The asset heavily utilizes the Component/ScriptableObject pattern to separate configuration data from the runtime logic.

2.3.1 ShipProperties.cs and StarshipProfileSO.cs

`ShipProperties` is the component on the starship prefab that holds the *runtime* flight parameters. `StarshipProfileSO` is the asset file containing the *template* data.

Design Philosophy: This separation allows `ShipProperties` to load data from a profile (via `LoadProfile()`) while still allowing its values to be modified independently during gameplay (e.g., power-ups) without altering the underlying asset file.

Configuration Workflow:

1. Create a new `StarshipProfileSO` asset (Create -> Navigation System -> Starship Profile).
2. Configure the flight parameters in the SO asset.
3. Assign the SO asset to the `profileSO` field on the `ShipProperties` component of the starship prefab, and ensure `useProfile` is checked.

Key Parameters (Highlights):

- **Core Physics:** `mass`, `maxSpeed`, `rotationDamping`, `angularDrag`.
- **Thrust/Rotation Power:** `forwardThrustPower`, `pitchPower`, etc.
- `maneuverabilityCurve` (AnimationCurve): Defines how agile (pitch/yaw) the ship is at different speeds. The X-axis is speed (m/s), and the Y-axis is a multiplier (0.0 to 1.0) applied to the base rotation power.
- **G-Force Settings:** `useGForceLimiter`, `maxOverallGForce`.

- **Boost System:** All parameters related to boost fuel, drain, recharge, and multiplier.
- **LERP Mode:** Parameters defining the simplified movement behavior (`lerpCruiseSpeed`, etc.).

Maneuverability Envelope (AI Helpers): `ShipProperties` includes fields (e.g., `MinTurnRadiusAtMaxSpeed`, `OptimalManeuveringSpeed`) and a helper method `CalculateMinTurnRadius(float speed)`. These define the physical limits of the ship's turning capability, used by advanced AI behaviors for predictive pathfinding.

2.4 Environment and Global Settings

The flight model is heavily dependent on the environment, particularly gravity and atmospheric density (drag).

2.4.1 EnvironmentManager.cs

The `EnvironmentManager` is a singleton (`Instance`) responsible for managing the global environmental settings of the scene.

Purpose: To centralize control over global physics parameters and provide a single source of truth for atmospheric drag coefficients used by all `AtmosphericStarshipController` instances.

Functionality:

- **Global Gravity:** The manager sets the global `Physics.gravity` vector based on the `gravityStrength` in the assigned profile.
- **Aerodynamic Drag:** The manager provides access to the base drag coefficients used in the custom drag model.

2.4.2 EnvironmentProfileSO.cs (ScriptableObject)

This asset defines the physical characteristics of the environment.

Key Parameters:

- `gravityStrength` (m/s²): The strength of gravity (e.g., 9.81 for Earth).
- **Base Aerodynamic Drag Coefficients (Linear and Quadratic):**
  - `baseForwardLinearDrag` / `baseForwardQuadraticDrag`
  - `baseSidewaysLinearDrag` / `baseSidewaysQuadraticDrag`

By changing the environment profile, developers can simulate flight in different atmospheres (e.g., thin atmosphere with low drag, or space with zero drag and zero gravity).

2.5 Sensory Systems and Obstacle Avoidance

The sensory system provides environmental awareness, crucial for AI navigation and physics effects like ground effect.

2.5.1 ShipSensorySystem.cs and SensorySettingsSO.cs

The `ShipSensorySystem` manages the ship's perception of the environment, focusing on altitude detection and obstacle avoidance. It uses a `SensorySettingsSO` for configuration.

Purpose: To detect obstacles, calculate avoidance vectors, and provide altitude information.

State Machine: The system operates a state machine to manage the scanning process efficiently:

1. **Searching (Default):** Passively checks altitude and probes forward for obstacles at defined frequencies.
2. **Avoiding:** Triggered when an obstacle is detected within `forwardCheckDistance`. The system initiates an active, detailed scan (defined by `coneAngle` and `coneRayCount`), analyzes the results, chooses the best escape route (`ObstacleAvoidanceVector`), and remains in this state for `avoidanceDuration`.
3. **Cooldown:** After avoiding, the system enters a brief cooldown (`cooldownDuration`) before returning to Searching.

Public API:

- `AltitudeAboveGround` (float): The current altitude.
- `IsObstacleAhead` (bool): True if in the Avoiding state.
- `ObstacleAvoidanceVector` (Vector3): The recommended direction to move (used by AI).
- `OnProximityAlert` (Event): Invoked when entering the Avoiding state.

Decision Logic (`ChooseBestAvoidancePath`): When analyzing the active scan results, the system scores potential escape routes. It applies biases defined in the settings: `upwardBiasBonus` (prefers climbing) and `turningInertiaBiasBonus` (prefers continuing the current turn direction) help select the most efficient path.

2.5.2 The Sensor Provider Pattern (ISensorProvider and SimpleSensorProvider)

The actual work of performing physics queries (Raycasts, SphereCasts) is abstracted away from the `ShipSensorySystem` into the `ISensorProvider` interface.

Design Philosophy (Provider Pattern): This pattern abstracts the *how* of sensory input from the *what* the system does with it. This allows swapping implementations based on performance needs without rewriting the core sensory logic. The `ShipSensorySystem` inspector allows selecting the `providerType`.

ISensorProvider Interface: Defines the contract for sensory operations, including starting/stopping passive scans (`StartPassiveScanning`, `StopForwardScanning`) and executing the immediate active scan (`PerformActiveAvoidanceScan`). The provider must invoke the `OnObstacleDetected` event when the passive forward check hits something.

SimpleSensorProvider.cs: This is the default implementation.

- **Mechanism:** It performs direct `Physics.Raycast` and `Physics.SphereCast` calls within its own `Update()` loop.

- **Frequency Management:** It interprets the `QueryFrequency` enum (e.g., `Every3Frames`, `Every_0_3_Seconds`) to manage its own update rate, staggering checks over time or frames to reduce performance spikes.

2.6 Weapons Systems

The weapon system provides the functionality for firing lasers, rockets (guided and unguided), and bombs.

2.6.1 ShipWeaponController.cs and WeaponStats.cs

The `ShipWeaponController` manages the firing logic, ammunition tracking, and projectile spawning. It uses `WeaponStats` ScriptableObjects to define the characteristics of each weapon type.

Purpose: To centralize weapon management, handle fire rate limits, ammo consumption, fire point iteration, and the complexities of gimbaling and guided weapon targeting.

Configuration:

- **WeaponStats (SO Assets):** Assigned to `primaryWeaponStats`, `secondaryWeaponStats`, and `bombardmentWeaponStats`.
- **Fire Points (Lists of Transforms):** `primaryFirePoints`, etc. Define the physical locations where projectiles are spawned.
- `maxGimbalAngle`: The maximum angle (degrees) that primary/secondary weapons can pivot towards the `AimPosition`.

Key Functionality:

- **Rate Limiting and Ammo:** Enforces `refireRate` and tracks ammunition.
- **Fire Point Cycling:** Iterates through the fire points to alternate firing locations.
- **Projectile Spawning:** Instantiates the `projectilePrefab` and initializes it, crucially passing the owner's velocity (`inheritedVelocity`) so the projectile inherits the ship's motion.

Gimbaling (Unguided Weapons): When firing an unguided weapon, the controller calculates the direction towards the `AimPosition`. It then rotates this direction towards the fire point's forward vector, clamped by `maxGimbalAngle`. This simulates gimbaled mounts. For lasers, an accuracy model (`ApplyAccuracyModel`) introduces spread based on the `WeaponStats`.

2.6.1.1 Guided Weapons Logic

Guided weapons utilize a "Fire-and-Forget" mechanism. The logic relies on the input state provided by the driver and the `CurrentTarget` (which is also set by the driver via `SetTarget()`).

Conditions for Firing (FireGuided Method):

1. The `WeaponStats.isGuided` must be true.
2. The `IShipDriver` must signal `IsTargetEngaged` (passed into `FireSecondary`).
3. A valid `CurrentTarget` must be set and alive.

4. The target must be within `maxLockDistance` and `maxLockAngle` relative to the ship's forward direction.

Launch and Handoff: If validated, the missile is launched. During `SpawnProjectile`, the controller detects if the prefab has a `GuidedRocketProjectile` component. If found, it calls `InitializeGuided()`, passing the `CurrentTarget` reference to the missile. The missile is then responsible for tracking the target independently.

If `isGuided` is true but the driver is not engaged, `FireSecondary` defaults to firing the weapon as unguided (dumb-fire) towards the `AimPosition`.

2.7 Health, Identity, and Lifecycle Management

Managing the state of ships—who they are, whether they are alive, and how they respawn—is handled by a cooperating set of components.

2.7.1 HealthComponent.cs and IHealthProvider.cs

The `HealthComponent` manages health and shield values, damage application logic, and crucially, acts as the **authoritative source for all damage and destruction audio-visual effects (FX)**.

Purpose: A universal system for managing durability, lifecycle triggers, and immediate combat feedback (impacts and explosions).

Damage Application Logic (`ApplyDamage`):

- Damage is first applied to `CurrentShields`. The `timeSinceLastDamage` timer is reset (interrupting regeneration).
- If shields are depleted, `OnShieldsBroken` is invoked, and remaining damage spills over to `CurrentHealth`.
- The `OnImpact` event is fired.
- **Internal FX:** Based on what was hit, the component automatically plays the assigned `shieldImpactVFX/SFX`, `shieldBreakVFX/SFX`, or `hullImpactVFX/SFX`.
- If `CurrentHealth` reaches zero, `Die()` is called.

Death and Lifecycle (`Die()`): When the ship reaches zero health, a highly coordinated destruction sequence begins to ensure FX play correctly before the ship is removed:

1. **Events Fired:** `OnDeathEvent` is immediately invoked. (If in Starship Mode, the `FactionManager` catches this and begins its respawn timer).
2. **Death FX:** The `deathExplosionVFX` and `deathSound` are triggered.
3. **Delayed Destruction:** To prevent the explosion from being abruptly cut off, the component starts the `DelayedDestructionCoroutine`.
4. **Hide & Wait:** All `Renderers` (except particle systems) and `Colliders` are instantly disabled so the ship "disappears," but the GameObject remains active for the duration of the `destructionDelay` (e.g., 3 seconds) to let the explosion finish.
5. **Deactivation:** Once the delay finishes, the GameObject is fully deactivated.

IHealthProvider Interface: Provides read-only access to the health state (`CurrentHealthNormalized`, `IsAlive`, etc.), used by AI, HUDs, and the `StarshipFXController`.

2.7.2 StarshipIdentity.cs

`StarshipIdentity` serves as the central identification tag for a ship within the simulation.

Purpose: To store the ship's allegiance, role, tactical parameters, and dynamic state. It is the primary component used by the `FactionManager` and AI systems for identification and lifecycle management.

Configuration:

- **Identity and Allegiance:** `FactionID`, `Role` (Defines the ship's team and tactical purpose).
- **Scene Patrol Configuration (Sentry):** `PatrolWaypoints` and `PatrolPattern`. Allows scene-specific patrol routes to be injected into the AI without changing the base prefab.
- **Lifecycle Management:**
    - `CanRespawn`: Controls whether the `FactionManager` will revive this ship after destruction.
    - `ResetWeaponsOnRespawn`: If true, ammo is fully replenished upon respawning.
    - `AllowDistanceBasedDeactivation`: **(Optimization)** If checked, the `FactionManager` will automatically deactivate this ship's GameObject when it travels too far from the player, drastically saving CPU resources. *Note: This should only be enabled for AI ships, never the Player.*
- **Tactical Distances:** `EngageDangerDistance`, `DisengageDangerDistance` (hysteresis for AI threat assessment), `MaxHomeDistance`, `ArrivalRadius`.

Interaction: `OnEnable`, `StarshipIdentity` automatically registers itself with the `FactionManager.Instance`. It also pre-calculates and caches squared versions of all tactical distances to ensure AI distance queries are highly optimized.

2.7.3 FactionManager.cs

The `FactionManager` is a critical singleton that acts as the central, authoritative directory for the entire simulation.

Purpose: To maintain accurate rosters of all ships, manage their lifecycle (activation, deactivation, respawning), and provide fast, optimized intelligence lookups.

Core Registries:

- **Master Lifecycle Registry (`_managedShips`):** A Dictionary tracking *all* ships, regardless of status, using the `ManagedShipState` class (which tracks `ShipStatus` - Active, Inactive, Destroyed - and `RespawnTimer`).
- **Active Rosters:** Optimized lists (`_allShipsRegistry`, `_leadersRegistry`, `_followersRegistry`) categorized by Faction. These registries **only contain Active ships** and are updated immediately when a ship's status changes.

Public API (Intelligence Queries): The manager provides efficient methods for AI to query the battlefield:

- `GetHostiles(Faction requesterFaction)`
- `GetFriendlyLeaders(Faction requesterFaction)`
- `GetHomeBases(Faction faction)`

Respawn Handling: The manager fully orchestrates the respawn process (see flow below).

2.7.4 Respawn Mechanics Flow

The respawn process is a coordinated effort:

1. **Death:** `HealthComponent.Die()` invokes `OnDeathEvent`.
2. **Registration:** `FactionManager.OnShipDestroyed` catches the event. It updates the ship's status to `Destroyed`, removes it from Active Rosters, and deactivates the ship's GameObject.
3. **Timer:** If `StarshipIdentity.CanRespawn` is true, the manager starts the `RespawnTimer`.
4. **Execution:** When the timer expires, `ExecuteRespawn` is called.
5. **Reposition:** The ship is moved to the designated respawn point (e.g., `TeamARespawnPoint`).
6. **Reset:** `AtmosphericStarshipController.ResetAndClearPhysicsState()` clears velocities. `HealthComponent.Initialize()` restores health/shields/ammo.
7. **Reactivation:** The GameObject is reactivated, the status is set to `Active`, and the ship is added back to the Active Rosters.

2.8 Visual Effects Controller

2.8.1 StarshipFXController.cs

The `StarshipFXController` is an optional component responsible for handling all visual (VFX) and sound (SFX) effects related to the starship's operation.

Purpose: To centralize FX management and decouple it from the core logic scripts. It is typically placed on a child GameObject (the visual model).

Functionality:

- **Continuous Effects (Update Loop):** Reads data like `CurrentThrustLevel` and `CurrentSpeed` every frame from the parent controller.
    - **VFX Modulation:** Modulates the emission rates of thruster ParticleSystems based on the thrust level.
    - **Audio Modulation:** Smoothly adjusts the volume and pitch of looping audio sources (engine, wind, boost) based on thrust and speed.
- **Event-Driven Effects (One-Shots):** Subscribes to events from `HealthComponent` (e.g., `OnImpact`, `OnDeathEvent`) and `ShipWeaponController` (e.g., `OnPrimaryFired`).
- **G-Force Feedback:** Listens to `OnGForceUpdate` and plays stress sounds if G-forces exceed thresholds.

Robustness: The controller is designed to gracefully handle missing AudioSources or ParticleSystems; the corresponding effect simply won't play, preventing errors.

2.9 Global State Management

2.9.1 GameStateManager.cs

The `GameStateManager` is a singleton that manages the overall application state, primarily focusing on pausing/resuming the game and enforcing cursor control.

Functionality:

- **Pause/Resume:** Manages the `CurrentState` (Playing/Paused) and typically sets `Time.timeScale` accordingly.
- **Cursor Control:** This is crucial for mouse-driven flight. The manager strictly enforces the cursor state every frame.
    - It interacts with the `IPlayerActivityProvider` interface (implemented by `PlayerShipDriver`).
    - If the player is actively playing, the cursor is set to `CursorLockMode.Locked` and `Cursor.visible = false`. This ensures uninterrupted mouse input for flight control.

Chapter 3: Player Setup and Experience

This chapter details the configuration and integration of the player-controlled starship. It focuses on the specific scripts responsible for handling user input, managing the camera perspective, displaying the Heads-Up Display (HUD), and configuring the audio-visual feedback unique to the player's experience.

While these components interact with the Core Starship Systems (like the AtmosphericStarshipController or HealthComponent), the focus here remains on the player-facing setup.

3.1. Overview of Player Architecture

The player experience is coordinated through several key components:

1. **GameInitializer:** Bootstraps the scene, ensuring the Input, Camera, and Player Driver are found and linked correctly.
2. **InputProvider:** A singleton manager that captures raw input based on customizable bindings.
3. **PlayerShipDriver:** The "pilot." It reads input, manages targeting/engagement, and translates these into control signals for the AtmosphericStarshipController.
4. **StarshipCameraController:** Manages the third-person camera view, following the ship smoothly and applying dynamic effects.
5. **StarshipHUDManager:** Reads data from the ship's systems and updates the UI elements.
6. **StarshipFXController:** Listens to events and state changes to trigger visual and audio feedback.

3.2. Scene Initialization

The setup process is automated by the GameInitializer script.

3.2.1. GameInitializer.cs

Purpose:

The GameInitializer handles the startup sequence. It ensures all necessary components for the player experience are present and correctly linked when the scene starts.

Setup:

Place a single instance of GameInitializer in your game scene, typically on a manager GameObject.

Configuration:

The script attempts to auto-detect components, but manual assignment is possible:

Field	Description
_playerCamera	(Optional) The main camera for the player. If left null, Camera.main (the camera tagged "MainCamera") will be used automatically.
_playerDriver	(Optional) The specific PlayerShipDriver instance in the scene. If left null, the script will search the scene for the first active PlayerShipDriver.
_inputProviderPrefab	(Optional) If no InputProvider is found in the scene, this prefab will be instantiated. If this is also null, an InputProvider component will be added to the GameInitializer GameObject as a fallback.

Interactions:

The initializer ensures an InputProvider exists, verifies the camera has a StarshipCameraController, finds the PlayerShipDriver, and then calls InitializePlayerView on the driver. This links the ship to the camera and signals the GameStateManager to lock the cursor for gameplay.

3.3. Input Management

The asset uses Unity's Input System, configured dynamically via the InputProvider. This allows customization of keyboard and mouse bindings directly in the Inspector.

3.3.1. InputProvider.cs

Purpose:

InputProvider is a singleton that centralizes input handling. It defines actions (Move, Boost, Fire) and maps them to specific keys defined in the Inspector.

Setup:

The GameInitializer ensures an InputProvider instance is present. It is accessible via InputProvider.Instance.

Configuration (KeyBindings Struct):

The primary configuration is done through the KeyBindings struct in the Inspector, allowing you to assign standard KeyCode values.

Binding Category	**Key Fields**	**Default**	**Description**
Starship Flight Controls	forwardThrust / backwardThrust	W/S	Controls primary acceleration/deceleration.
	leftThrust / rightThrust	A/D	Used for keyboard-based Yaw (turning).
	verticalThrustUp / Down	Space/LCtrl	Controls vertical movement (VTOL).
	rollLeft / rollRight	Q/E	Controls manual rolling.
	boost	LShift	Activates the boost system.
Weapon Controls	firePrimary	Mouse0 (LMB)	Fires the primary weapon (e.g., lasers).
	fireSecondary	F	Fires the secondary weapon (e.g., rockets).
	fireBomb	G	Drops bombardment weapons.

	aimMode	Mouse1 (RMB)	**Crucial:** Hold to engage the nearest hostile target within the targeting cone (enables auto-aim and missile lock).
System & Action Controls	zoomIn / zoomOut	Z/X	Controls camera zoom distance.
	pause	Escape	Toggles the pause menu.

How It Works:

At runtime, InputProvider reads these KeyCode settings and automatically sets up the corresponding InputAction bindings in the Input System. This provides the flexibility of Inspector-based configuration without needing to edit complex Input Action Assets.

3.4. Player Control Logic

The PlayerShipDriver translates the player's input into actions and manages the targeting system.

3.4.1. PlayerShipDriver.cs

Purpose:

The PlayerShipDriver acts as the pilot. It implements IShipDriver to control the AtmosphericStarshipController. It reads data from the InputProvider, determines desired movement, manages target engagement, and calculates the aiming point for weapons. It also implements IPlayerActivityProvider to manage cursor locking via the GameStateManager, ensuring the cursor is hidden and locked during flight.

Setup:

This script must be attached to the player's starship GameObject, alongside core components like AtmosphericStarshipController, ShipProperties, ShipWeaponController, and StarshipIdentity.

Configuration:

Field	Description
maxAimDistance	The maximum distance (in meters) used for calculating the aim point when *not* engaged. This should align with your primary weapon's maximum range (e.g., 1000m).

targetingConeAngle	The angle (in degrees) defining the cone in front of the ship where targets can be automatically engaged when the aimMode button (RMB) is held. A value of 15-25 degrees is typical.

Interactions and Unified Targeting:

1. **Input Reading:** Reads movement and action inputs from the InputProvider. Mouse input is scaled by the mouseSensitivity defined in the attached ShipProperties.
2. **Target Engagement:**
    - When the player holds the aimMode button (RMB), the driver queries the FactionManager for hostiles within the targetingConeAngle.
    - If a target is found, IsTargetEngaged becomes true.
    - The driver activates the playerTargetVisualizer GameObject on the target (configured on the target's HealthComponent) for visual confirmation.
3. **Aim Calculation:**
    - If engaged, the AimPosition sent to the controller is the exact location of the target. This provides auto-aim assistance for unguided weapons.
    - If not engaged, the AimPosition is calculated by projecting a ray from the center of the camera view out to maxAimDistance.
4. **Weapon Control:** The IsTargetEngaged state and the currentTarget are passed to the ShipWeaponController. If engaged, guided secondary weapons will lock onto the target when fired.

3.5. Camera System

The StarshipCameraController provides a robust, smooth, and dynamic third-person perspective.

3.5.1. StarshipCameraController.cs

Purpose:

This script manages the camera's position and rotation relative to the player's ship. It handles smooth following, collision detection, zoom, and dynamic effects based on the ship's movement (acceleration lag and G-force feedback).

Setup:

Attach this script to your main Camera GameObject. The GameInitializer ensures this happens and links it to the player ship via the Target field. The camera automatically subscribes to the FactionManager's OnPlayerShipRespawned event to handle seamless transitions during respawns.

Configuration (CameraSettings Struct):

Positioning and Zoom

Field	Description

anchorOffset	The point relative to the ship's origin that the camera orbits (e.g., Y=2.5 places it slightly above the ship).
defaultDistance	The standard distance the camera maintains from the anchor point.
minDistance / **maxDistance** / **zoomSpeed**	The limits and speed for the camera zoom functionality.

Smoothing and Rotation

Field	Description
rotationSmoothSpeed	How quickly the camera rotation catches up to the target's orientation. Higher values (15-25) provide a tight, responsive feel. Lower values (5-10) create a looser, cinematic follow.

Auto-Alignment and Leveling

These settings control how the camera automatically reorients itself behind the ship.

Field	Description
returnToHorizonSpeed	The speed at which the camera's pitch levels out (returns to 0) relative to the ship's orientation.
autoAlignYaw	If enabled, the camera will automatically rotate horizontally to align directly behind the ship.
autoAlignSpeed	Defines the speed of the realignment.

Dynamic Effects (Acceleration Lag)

This effect causes the camera to subtly fall back when the ship accelerates rapidly, enhancing the sense of speed.

Field	Description

accelerationLagAmount	The maximum distance the camera can lag behind during full thrust/boost.
accelerationLagSpeed	How quickly the camera transitions into and out of the lagged position.

G-Force Effects

Simulates the physical strain of high-G maneuvers through camera shake and FOV changes.

Field	Description
gForceEffectThreshold	The G-force level (e.g., 5G) at which the effects begin.
gForceShakeAmount	The maximum intensity of the procedural camera shake applied at high Gs.
gForceFovChange	The maximum change in Field of View applied at high Gs (simulating tunnel vision or strain).
gForceEffectSpeed	How smoothly the effects ramp up and down as G-forces change.

Collision Detection

Prevents the camera from clipping through terrain or objects.

Field	Description
collisionEnabled	Toggles the camera collision system.
collisionLayers / **collisionRadius** / **collisionOffset**	Configures the physics SphereCast used to detect obstacles and the buffer distance.

3.6. Heads-Up Display (HUD)

The StarshipHUDManager connects the game's UI elements to the player ship's data.

3.6.1. StarshipHUDManager.cs

Purpose:

This script updates the various Text (TextMeshPro), Slider, and Image components on the HUD based on the state of the player's ship components.

Setup:

Attach this script to your main HUD Canvas or a panel within it. It automatically detects the active PlayerShipDriver and links to the ship it controls.

Configuration:

Configuration involves linking the required UI elements in the Inspector.

Category	UI Elements	Description
Flight Data	velocityText, altitudeText, gForceText	Displays speed, altitude (ALT and AGL), and G-Force.
Thrust/Power	thrustGaugeSlider/Image, verticalThrustSlider/Image	Visualizes main and vertical thrust levels. Fill colors change based on direction (configured in the UI Colors section).
	boostGaugeSlider	Displays remaining boost energy.
Health	healthBarSlider, shieldGaugeSlider	Displays hull integrity and shield strength. Shield gauge hides if the ship has no shields.
Weapons/Ammo	primaryAmmoText, secondaryAmmoText, crateCountText (Bombs)	Displays ammunition counts or "∞" if infinite.
Targeting/Status	crosshairImage	The fixed center-screen crosshair image.

	targetStatusText	Displays "ENGAGED" when the PlayerShipDriver has locked a target.
	proximityWarningText	Displays collision warnings from the ShipSensorySystem.

UI Colors:

Customize the colors for gauges and warnings (e.g., forwardThrustColor, warningColor) directly in the Inspector.

3.7.1 StarshipFXController.cs

The `StarshipFXController` provides a comprehensive, centralized system for managing **continuous operational feedback** (engines, atmospheric drag) and **weapon firing effects**.

(Note: Damage impacts and death explosions are handled exclusively by the `HealthComponent`. This ensures that critical destruction FX always play reliably, even if the FX controller is disabled or interrupted).

Purpose: To decouple atmospheric, engine, and weapon audio/visuals from the core logic scripts. It modulates parameters dynamically based on the ship's current physical state.

Setup: It is recommended to attach the `StarshipFXController` to a child GameObject of the main starship prefab (e.g., the visual model), rather than the root. It automatically finds the required core components (`AtmosphericStarshipController`, `HealthComponent`, `ShipWeaponController`, `ShipSensorySystem`) in its parent hierarchy.

Configuration:

Audio Sources: The system requires multiple AudioSource components for proper mixing. If an AudioSource is missing, the script will attempt to dynamically add one for the `oneShotAudioSource`, but setting them up manually is recommended:

- `engineAudioSource`: Main forward engine loop. Pitch and volume are modulated by thrust.
- `boostAudioSource`: Boost overlay loop. Fades in/out smoothly.
- `reverseThrusterAudioSource` & `verticalThrusterAudioSource`: Loops for maneuvering thrusters.
- `movementAudioSource`: Wind/atmospheric noise. Modulated by the ship's current speed.
- `oneShotAudioSource`: Handles transient sounds like weapon fire, proximity alerts, and G-force stress.

Visual Effects (VFX):

- **Engine VFX:** Lists of Particle Systems (`mainThrustersVFX`, `reverseThrustersVFX`, `verticalThrustersVFX`). The controller caches their default `rateOverTime` and dynamically modulates emission rates based on the corresponding thrust level.
- **Boost Specific VFX:** Optional VFX (`boostSpecificVFX`) activated only during boost.
- **Movement VFX:** `speedLinesVFX` (activated above the `speedLinesActivationThreshold`).
- **Weapon VFX (Prefabs):** `primaryMuzzleFlashPrefab` and `secondaryMuzzleFlashPrefab`. These prefabs are instantiated dynamically at the weapon fire points when fired.

Audio Clips (SFX): Assign specific AudioClip assets for all actions, including loops for engines (`mainEngineLoopClip`, `reverseThrusterLoopClip`, etc.), movement (`windLoopClip`), weapon firing sounds (`primaryFireClip`, `secondaryFireClip`, `bombDropClip`), system alerts (`proximityAlertClip`), and `gForceStressClips` (played during high-G maneuvers).

Functionality (The Update Loop): Every frame, the controller reads data like `CurrentThrustLevel`, `CurrentSpeed`, and `IsTryingToBoost` from the parent physics controller to seamlessly scale particle emission rates and audio volumes. It also monitors the `HealthComponent`; if the ship dies, the FX controller instantly cuts off all continuous engine and wind loops.

Creating FX Variations: To create variations (e.g., different engine colors or sounds for different factions):

1. Create a prefab variant of the GameObject containing the `StarshipFXController`.
2. In the variant, swap out the Particle Systems and Audio Clips in the Inspector.
3. Use this variant in the corresponding starship prefab.

3.7.2. Performance Recommendations: Player vs. AI

The StarshipFXController enables rich feedback, but running complex FX on numerous ships can impact performance (CPU overhead for Audio Sources, CPU/GPU load for Particle Systems).

Player Ship (High Immersion):

It is highly recommended to utilize the full suite of effects on the player's ship. Enable all thruster VFX, detailed impact sounds, G-force stress sounds, weapon effects, and environmental audio for maximum immersion.

AI Ships (Optimized Performance):

For AI ships, it is crucial to use a simplified FX setup to maintain performance and avoid overwhelming the audio mix.

- **VFX Simplification:** Consider enabling only the mainThrustersVFX. Disable reverse/vertical thrusters, speed lines, and complex impact VFX.
- **SFX Simplification:** Use only essential audio. Disable G-force sounds, wind noise, and most impact/alert sounds.
- **Reduce Audio Sources:** If an Audio Source (like boostAudioSource) is unused in an optimized AI profile, ensure the reference is null or the component is removed from the AI FX prefab variant to save resources.

By creating distinct Player and AI FX configurations, you can balance immersion with performance in large-scale scenarios.

3.8. Pause Menu

The PauseMenuController manages the game's pause state and UI.

3.8.1. PauseMenuController.cs

Purpose:

This script listens for the pause input from the InputProvider and communicates with the GameStateManager to stop time and manage cursor visibility. It also handles the visibility of the pause menu UI itself.

Setup:

Attach this script to your Pause Menu UI structure (e.g., the root panel).

Configuration:

- **pauseMenuRoot:** The root GameObject of the pause menu UI. This will be activated/deactivated based on the pause state.
- **resumeButton** / **restartButton:** UI Buttons linked to the resume and restart actions in the GameStateManager.

3.9. Summary of Component Interactions

- **GameInitializer** links InputProvider, PlayerShipDriver, and StarshipCameraController.
- **InputProvider** captures raw input based on KeyBindings.
- **PlayerShipDriver** reads from InputProvider, calculates AimPosition using the camera view, manages engagement via FactionManager, and controls the AtmosphericStarshipController. It also informs GameStateManager that the player is active (locking the cursor).
- **StarshipCameraController** follows the ship, applying dynamic effects based on data from AtmosphericStarshipController (thrust, G-force).
- **StarshipHUDManager** reads data from all core systems and the driver to update the UI.
- **StarshipFXController** listens to events and state changes from core systems to trigger VFX and SFX.
- **PauseMenuController** interacts with InputProvider and GameStateManager to halt gameplay.

Chapter 4: AI Pilot System

4.1. Introduction

The AI Pilot System provides a comprehensive, modular framework for controlling starships within the asset package. It is designed to be flexible, allowing developers to create everything from simple patrol drones to complex tactical combatants.

This chapter details the architecture, configuration, and implementation of the AI components. It is intended for Unity developers who need to understand how the AI makes decisions, how it flies the ships, and how to customize its behavior for their specific game needs.

While the AI interacts with core systems like `AtmosphericStarshipController` and `FactionManager`, this chapter assumes those systems are documented elsewhere and focuses exclusively on the AI-specific logic.

4.2. AI Architecture Overview

The AI system employs a decentralized architecture, separating high-level decision-making from low-level execution. This separation enhances modularity and makes the system easier to debug and extend.

4.2.1. Key Components

The AI architecture can be conceptualized as having a "Mind," "Hands," "Eyes," and "Trigger Finger."

1. **AiBrain (The Mind):** This component is responsible for high-level strategic decisions. It assesses the environment, evaluates potential actions based on predefined rules, manages targets, and selects the appropriate behavior for the situation.
2. **SimpleAiPilot (The Hands):** This component handles the execution of flight maneuvers. It receives a `NavigationGoal` from the active behavior and translates it into the raw inputs (Pitch, Yaw, Roll, Thrust) required by the `AtmosphericStarshipController`. It implements the `IShipDriver` interface, acting just like a human player.
3. **AiWeaponController (The Trigger Finger):** Operating independently of the `SimpleAiPilot`, this component manages aiming and firing. It calculates lead targeting, evaluates firing solutions, and decides when to use primary and secondary weapons based on the `AiBrain`'s attack target.
4. **SituationalAwarenessSystem (SAS) (The Eyes):** While not a component on the ship itself, the SAS is a static utility system that acts as the AI's primary source of intelligence. The `AiBrain` queries the SAS to find hostiles, allies, objectives, and assess danger levels, relying on data centralized in the `FactionManager`.
5. **Behaviors (ISimpleAiBehavior):** These are modular classes defining specific tasks (e.g., Patrol, Attack, Flee). The `AiBrain` selects and activates a behavior, which then provides the `SimpleAiPilot` with moment-to-moment navigation goals.
6. **Configuration Assets (ScriptableObjects):**
    - **AiActionSO & AiConditionSO:** Define the rules and priorities that govern the `AiBrain`'s decisions.
    - **FlightPersonalitySO:** Tunes the `SimpleAiPilot`'s flight characteristics (e.g., aggressive vs. precise).
    - **BehaviorConfigSO (e.g., AttackConfigSO, SentryConfigSO):** Tunes the parameters for specific behaviors.

4.2.2. Data Flow

The typical flow of information during AI operation is as follows:

1. **Sense:** The `AiBrain` wakes up during its "Think Cycle" and queries the `SituationalAwarenessSystem` (SAS) to update its understanding of the environment (e.g., "Am I in danger?", "Where is the closest hostile?").
2. **Decide:** The `AiBrain` evaluates its prioritized list of `AiActionSO`s. It checks the `AiConditionSO`s associated with each action.

3. **Act (High Level):** When an action is selected, the `AiBrain` executes the action's targeting logic (setting `MoveTarget` and/or `AttackTarget`). It then instantiates the corresponding `ISimpleAiBehavior` (using the `BehaviorConfigSO` factory) and assigns it to the `SimpleAiPilot`.
4. **Execute (Low Level - Flight):** Every frame, the `SimpleAiPilot` asks the active behavior for a `NavigationGoal` (target position and speed). The pilot then uses its `FlightPersonalitySO` settings (e.g., PID controllers) to calculate the necessary `ShipInputState` (Pitch, Yaw, Roll, Thrust) to reach that goal.
5. **Execute (Low Level - Combat):** Simultaneously, the `AiWeaponController` uses the `AiBrain`'s `AttackTarget` to calculate the `AimPosition` and determines if weapons should be fired.
6. **Control:** The `AtmosphericStarshipController` gathers the complete `ShipInputState` (aggregated from the Pilot and Weapon Controller) and applies the forces to the Rigidbody.

4.3. AiBrain: The Decision Maker

The `AiBrain` component is the strategic core of the AI. It uses a prioritized-list Utility AI approach for decision-making.

4.3.1. The Think Cycle

The `AiBrain` does not evaluate decisions every frame. Instead, it operates on a "Think Cycle" (configurable via `ThinkCycleInterval`, typically 1-3 seconds). This improves performance and introduces a natural reaction delay, preventing the AI from appearing omniscient.

4.3.2. Prioritized Actions (AiActionSO)

The AI's personality is defined by the `Actions` list in the `AiBrain` inspector. This is a list of `AiActionSO` ScriptableObjects, evaluated top-down.

When the `AiBrain` thinks, it iterates through the list. The first action whose conditions are met and whose probability check succeeds is selected. This prioritized list allows developers to define strict hierarchies (e.g., "Always Flee if health is critical, otherwise Attack if enemies are present, otherwise Patrol").

4.3.2.1. AiActionSO Configuration

An `AiActionSO` defines a tactic or response:

- **BehaviorToActivate:** The `AiMode` (e.g., Attack, Sentry, Gofer) that the AI should switch to if this action is chosen.
- **Conditions:** A list of `AiConditionSO`s. All conditions must be true for the action to be considered.
- **Behavioral Inertia (Probability):**
    - `EnterProbability`: The chance (0-1) the AI will transition *into* this action when its conditions are met.
    - `StayProbability`: The chance (0-1) the AI will remain *in* this action during the next think cycle, provided the conditions are still met. This prevents rapid switching and adds "stickiness" to behaviors.

- **Time Constraints:** `ActiveDuration` forces the AI to stay in the action for a minimum time. `CooldownDuration` prevents the action from being re-selected immediately after exiting.
- **Targeting:** Defines how the AI acquires a target when this action starts (see Section 3.4).

4.3.3. Conditions (AiConditionSO)

Conditions are the "rules" the AI uses to sense its environment. They are reusable ScriptableObjects. Examples include:

- **HealthConditionSO:** Checks if Health or Shields are above/below a threshold.
- **ProximityConditionSO:** Checks the distance to specific types of targets (Hostile, Leader, Home Base).
- **TargetConditionSO:** Checks if the AI currently has a valid MoveTarget or AttackTarget.
- **AltitudeConditionSO:** Checks if the ship is flying too high or too low.
- **RoleConditionSO:** Checks the ship's assigned role (Leader, Follower).

All conditions utilize the `SituationalAwarenessSystem` (SAS) or access data directly from the `AiBrain`'s cached references (like `HealthProvider` or `SensorySystem`).

4.3.4. Decision Logic (The Two-Phase Check)

During the Think cycle, the `AiBrain` uses a two-phase check to balance responsiveness with stability:

1. **Phase 1 (Stability Check):** It first checks if the `_currentAction` should be maintained. If the `ActiveDuration` hasn't expired, or if the conditions are still met and the `StayProbability` check passes, the current action is marked as the `preliminaryWinner`.
2. **Phase 2 (Priority Check):** The brain then iterates through the `Actions` list, starting from the top (highest priority). It only checks actions that have a higher priority than the `preliminaryWinner`. If a higher priority action's conditions are met and its `EnterProbability` check passes, it overrides the `preliminaryWinner`.

This system ensures that high-priority actions (like emergency evasion or reacting to immediate threats) can interrupt lower-priority actions (like patrolling), while the inertia system prevents oscillation between similar actions.

4.3.5. Target Management

The `AiBrain` manages two primary target slots:

- **MoveTarget:** The destination for navigation behaviors (e.g., Gofer, Loiterer).
- **AttackTarget:** The target for combat behaviors (BehaviorAttack) and the `AiWeaponController`.

4.3.5.1. Target Acquisition

When an `AiActionSO` is activated, it can specify a `TargetAcquisitionType` to automatically find and assign a target to a slot.

Important: When the `AiBrain` transitions to a *new* action, both `MoveTarget` and `AttackTarget` are unconditionally cleared. The new action is entirely responsible for setting the targets it needs.

Common Acquisition Types:

- `FindClosestHostile`: Queries SAS for the nearest enemy.
- `FindNearestHomeBase`: Queries SAS for the nearest friendly base.
- `FindClosestFriendlyLeader`: Queries SAS for the nearest ally marked as a Leader.
- `FindRandomTeamWaypoint`: Selects a random waypoint assigned to the faction globally.
- `FindPlayer`: Directly targets the player character.

4.3.6. Behavior Instantiation (The Factory Pattern)

The `AiBrain` does not directly contain the logic for behaviors. Instead, it uses a Factory Pattern mediated by `BehaviorConfigSO` assets.

The `Behavior Configurations` list on the `AiBrain` maps an `AiMode` (e.g., `AiMode.Attack`) to a specific configuration asset (e.g., `AttackConfigSO_Aggressive`).

When an action activates an `AiMode`, the `AiBrain` looks up the corresponding `BehaviorConfigSO`. It then calls the `CreateBehavior()` method on that asset. This method creates a new instance of the behavior class (e.g., `new BehaviorAttack(thisConfig)`) and returns it.

This pattern ensures that behaviors are stateless until activated and are initialized with the correct configuration parameters defined in the asset.

4.4. SimpleAiPilot: Flight Execution

The `SimpleAiPilot` component is responsible for the moment-to-moment control of the starship. It bridges the gap between the high-level goals of the `AiBrain`/Behaviors and the physics-based `AtmosphericStarshipController`.

4.4.1. Role as IShipDriver

`SimpleAiPilot` implements the `IShipDriver` interface. This allows it to be assigned to the `AtmosphericStarshipController` just like a `PlayerShipDriver`. The controller requests the `ShipInputState` (Pitch, Yaw, Roll, Thrust, Boost, etc.) from the pilot every frame.

4.4.2. The NavigationGoal

The pilot receives its instructions via a `NavigationGoal` struct, updated every frame by the active `ISimpleAiBehavior`.

- **TargetPosition:** The world-space position the pilot should fly towards.
- **DesiredSpeed:** The speed the pilot should attempt to maintain. If null, the pilot flies at maximum speed (adjusted by `FlightPersonalitySO`).
- **ArrivalTolerance:** The distance at which the goal is considered reached.
- **SlowDownRadius:** The distance at which the pilot should begin decelerating to arrive smoothly at the target.

4.4.3. Flight Styles and Personality

The `SimpleAiPilot` uses a `FlightPersonalitySO` asset to determine its flight characteristics. The core of the personality is the `FlightStyle`:

- **Calculated (PID Control):** Uses Proportional-Integral-Derivative (PID) controllers for smooth, precise, and stable flight. This style is excellent for accurate maneuvering and formation flying but can sometimes appear robotic. It requires careful tuning of the PID gains in the `FlightPersonalitySO`.
- **Focused (Proportional Control):** Uses simpler proportional control for rotation. This style prioritizes aligning with the target before applying significant thrust. It looks more decisive but can overshoot targets if `RotationAggressiveness` is too high.
- **Reckless (Proportional Control):** Similar to Focused, but applies full thrust constantly, even while turning ("turning and burning"). This is the fastest but least precise style.

4.4.4. Understanding PID Controllers (Calculated Style)

When using the `Calculated` style, the pilot uses three independent `UtilityPIDController` instances for Pitch, Yaw, and Roll stabilization.

A PID controller attempts to minimize the "error" (the difference between the current state and the desired state). In flight terms, the error is the angle between the ship's forward vector and the `TargetPosition`.

- **Proportional (P):** Applies a correction proportional to the current error. A large error results in a strong correction. This provides the main rotational force.
- **Integral (I):** Accumulates error over time. This helps overcome persistent external forces and ensures the ship eventually reaches the exact target orientation, eliminating steady-state error.
- **Derivative (D):** Responds to the *rate of change* of the error. This acts as a damper, slowing the rotation as it approaches the target to prevent overshoot and oscillation.

Tuning these gains is crucial for the `Calculated` style (see Section 6.1).

4.4.5. Proportional Control (Focused/Reckless Styles)

In `Focused` and `Reckless` styles, the rotation inputs are calculated simply by multiplying the error angle by the `RotationAggressiveness` factor. This is faster to calculate than PID but lacks the stability and precision provided by the Integral and Derivative terms.

4. 4.6. Thrust Management and Boosting

The pilot calculates the required thrust based on the `DesiredSpeed` and the current speed (`speedError`).

- **Acceleration:** The amount of thrust applied depends on the `FlightStyle`. `Calculated` and `Focused` styles reduce thrust if the ship is not aligned with the target, while `Reckless` applies full thrust regardless of alignment.
- **Deceleration:** If `DesiredSpeed` is lower than the current speed, the pilot applies reverse thrust (active braking).

- **Boosting:** If allowed by the `FlightPersonalitySO`, the pilot will activate the boost if the `DesiredSpeed` is high, the ship is aligned with the target, and the `speedError` is positive (needs to accelerate).

4.4.7. Safety Overrides (Altitude Maintenance)

The `SimpleAiPilot` includes a safety override system (`MaintainMinimumAltitude`) that operates independently of the active behavior if enabled in the `FlightPersonalitySO`.

It uses data from the `ShipSensorySystem` to check the `AltitudeAboveGround`. If the altitude is below the `MinimumAltitude` threshold, the pilot applies corrective vertical thrust (up or down, depending on the ship's orientation) to push away from the terrain. This override takes precedence over the vertical thrust input calculated by the behavior.

4.4.8. Obstacle Avoidance (Interaction)

The `SimpleAiPilot` provided in this context does *not* implement direct obstacle avoidance steering. While the `ShipSensorySystem` detects obstacles and calculates an `ObstacleAvoidanceVector`, the provided behaviors (Attack, Sentry, etc.) do not currently utilize this vector to modify their `NavigationGoal`. Implementing full obstacle avoidance steering would require modifying the behaviors to integrate this sensory data.

4.5. AiWeaponController: Combat Management

The `AiWeaponController` handles all aspects of weapon usage for the AI. It operates in its own `Update` loop, decoupled from the flight maneuvers executed by `SimpleAiPilot`.

4.5.1. Targeting and Aim Calculation

The controller reads the `AttackTarget` from the `AiBrain`. If a valid target exists, it calculates the required aim point.

4.5.1.1. Interception Calculation (Leading the Target)

To accurately hit a moving target, the AI must lead its shots. The `AiWeaponController` uses the `InterceptionCalculator` utility class to perform a first-order interception calculation. This calculation considers the shooter's position and velocity, the target's position and velocity, and the projectile speed of the primary weapon.

The result is the `AimPosition`—the world-space point where the projectile and target will theoretically collide.

4.5.1.2. Aim Error

To simulate pilot skill levels, the `AiWeaponController` applies an "Aim Error" based on the configuration (`CombatPersonalitySO` or `AttackConfigSO`). The `ErrorScale` parameter defines how much the aim point deviates from the perfect interception point, with the error increasing with distance.

4.5.2. Firing Logic

The `AiWeaponController` determines when to fire based on several factors:

1. **Firing Solution:** The controller evaluates if a valid firing solution exists. This requires the target to be within the weapon's `maxRange` (modified by `RangeMultiplier`) and the `AimPosition` to be within the `FiringConeAngle` of the ship's forward vector.
2. **Primary Weapons (Burst Control):** Primary weapons (like lasers) use a burst firing mechanism. `Min/MaxBurstDuration` controls how long the AI fires continuously, and `Min/MaxBurstCooldown` controls the pause between bursts. This simulates heat management and prevents predictable firing patterns.
3. **Secondary Weapons (Rockets/Missiles):** Secondary weapons have a `SecondaryFireRate` (probability per second) and a `SecondaryWarmupDelay` (minimum time engaged before use).
    - **Unguided:** Require a valid firing solution (similar to primary weapons).
    - **Guided:** If the weapon is guided (`isGuided` in `WeaponStats`), the firing solution check is often bypassed, relying instead on the `ShipWeaponController`'s lock constraints, provided the AI is actively engaged.

The `SimpleAiPilot` reads the `FirePrimary`, `FireSecondary`, and `AimPosition` outputs from the `AiWeaponController` and includes them in the final `ShipInputState` sent to the `AtmosphericStarshipController`.

4.6. Configuring AI Personalities

The AI's behavior is heavily influenced by ScriptableObject configuration assets.

4.6.1. FlightPersonalitySO

The `FlightPersonalitySO` defines how the ship maneuvers.

4.6.1.1. Core Style

- **Style:** Selects the control method (Calculated, Focused, Reckless). See Section 4.3.

4.6.1.2. General Movement Parameters

- **AlignmentConeAngle:** The "deadzone" angle (degrees). If the target is within this cone, the AI stops rotating. Used to prevent wobble. (Typical range: 2.0 - 5.0).
- **SpeedMultiplier:** Multiplier applied to the ship's base max speed. (e.g., 0.8 for cautious pilots, 1.2 for aggressive ones).
- **AllowBoost:** Whether this personality permits the use of boost.

4.6.1.3. Focused/Reckless (Proportional Control) Parameters

- **RotationAggressiveness:** The proportional gain (P) for rotation. Higher values mean snappier turns but more overshoot. (Typical range: 3.0 - 8.0).
- **FocusedThrustThresholdAngle:** In Focused mode, the angle at which the AI starts applying significant thrust. A smaller angle means the AI waits until it is precisely aligned before accelerating.

4.6.1.4. Calculated (PID Control) Parameters

These gains require careful tuning based on the ship's mass and torque capabilities.

- **Pitch/Yaw Kp (Proportional Gain):** How strongly the AI reacts to the current error.
- **Pitch/Yaw Ki (Integral Gain):** How strongly the AI corrects accumulated errors over time.
- **Pitch/Yaw Kd (Derivative Gain):** How strongly the AI dampens the rotation as it approaches the target.
- **IntegralClamp:** The maximum value for the integral term (I) to prevent windup. Usually set to 1.0.

4.6.1.5. Terrain Avoidance

- **UseMinimumAltitude:** Enables the safety override system.
- **MinimumAltitude:** The desired minimum distance (meters) to keep from the ground.

4.6.1.6. Upright Stabilization

- **MaintainUprightOrientation:** Enables cosmetic rolling to stay upright.
- **Roll Kp/Ki/Kd:** PID gains specifically for the upright stabilization roll controller.

4.6.2. Combat Configuration (AttackConfigSO)

Combat configuration parameters are primarily consolidated within the `AttackConfigSO` when the `BehaviorAttack` is active. This asset governs both the flight maneuvers during combat and the weapon usage (as it includes all fields typically found in a `CombatPersonalitySO`).

4.6.2.1. Maneuvering Style

- **Evasiveness:** The probability (0-1) that the AI will initiate an evasive maneuver during combat.
- **PreferredEngagementRange:** The ideal distance the AI tries to maintain during the `Dogfighting` state.
- **EngagementRangeTolerance:** The buffer around the preferred range. If the distance exceeds the preferred range plus tolerance, the AI switches back to the `Engaging` state.

4.6.2.2. Accuracy and Weapon Control

These parameters are utilized by the `AiWeaponController` when initialized with the configuration from the `AttackConfigSO`.

- **FiringConeAngle, ErrorScale, RangeMultiplier:** Defines the AI's accuracy and engagement envelope.
- **Burst Control (Min/Max Duration/Cooldown):** Defines primary weapon firing patterns.
- **Secondary Fire Rate/Warmup:** Defines secondary weapon usage probability and delay.

4.7. The Behavior System (ISimpleAiBehavior)

The behavior system provides modular, reusable tasks for the AI. The `ISimpleAiBehavior` interface defines the contract that all behaviors must follow: `Initialize`, `UpdateGoal`, `OnExit`, and `GetName`.

4.7.1. Common Behaviors

4.7.1.1. BehaviorIdle

- **Goal:** Do nothing.
- **Logic:** Returns a `NavigationGoal` with `DesiredSpeed` set to 0 at the current position, causing the pilot to brake and stop.

4.7.1.2. BehaviorGofer

- **Goal:** Fly directly to the `AiBrain.MoveTarget`.
- **Logic:** Returns a `NavigationGoal` pointing at the `MoveTarget`'s current position. It uses the `SlowDownRadius` and `ArrivalTolerance` defined in `GoferConfigSO`.

4.7.1.3. BehaviorSentry

- **Goal:** Patrol a sequence of waypoints defined specifically on the ship's `StarshipIdentity` component in the scene.
- **Logic:** This behavior reads the `PatrolWaypoints` and `PatrolPattern` (Loop, Sequential, Random) directly from the `StarshipIdentity`. The `SentryConfigSO` factory retrieves this scene-specific data during instantiation.

4.7.1.4. BehaviorTeamPatrol

- **Goal:** Patrol a sequence of global waypoints defined for the faction in the `FactionManager`.
- **Logic:** Similar to Sentry, but it retrieves the waypoint list from the `FactionManager`. It typically uses a Sequential pattern, optimizing the route by starting at the nearest waypoint.

4.7.1.5. BehaviorLoiterer

- **Goal:** Fly randomly within a defined radius around a center point.
- **Logic:** The center point is determined by `AiBrain.MoveTarget`. If `MoveTarget` is null, it loiters around the position where the behavior was first activated. It periodically selects a new random `TargetPosition` within the `Radius` defined in `LoiterConfigSO`.

4.7.1.6. BehaviorAltitudeCorrection

- **Goal:** Move the ship into a safe altitude zone if it is flying too high or too low.
- **Logic:** Reads the `AltitudeAboveGround` from the `ShipSensorySystem` and generates a `NavigationGoal` to move vertically towards the safe zone defined in `AltitudeCorrectionConfigSO`.

4.7.1.7. BehaviorAttack

`BehaviorAttack` is responsible for executing combat maneuvers. It uses a state machine internally to manage the flow of combat.

- **Engaging (ManeuverEngage):** Closes the distance to the target until within the `PreferredEngagementRange`.

- **Dogfighting (ManeuverDogfight):** Attempts to maintain the preferred range while maneuvering for advantage (strafing, circling, or tailing).
- **Evading (ManeuverEvade):** Executes a short, sharp defensive maneuver if the `Evasiveness` check passes.

4.7.1.8. BehaviorFlee

- **Goal:** Retreat to a designated safe location provided by the `AiBrain`.
- **Logic:** Reads the destination exclusively from `AiBrain.MoveTarget`. If no target is provided, the ship idles. Otherwise, it flies toward the target at maximum speed until it reaches the `ArrivalTolerance` defined in the `FleeConfigSO`.

4.7.1.9. BehaviorTakeoff

- **Goal:** Ascend vertically to a designated safe altitude before resuming normal AI operations.
- **Logic:** Ignores the `AiBrain.MoveTarget`. Instead, it calculates a target position directly above the ship's current location based on the `TakeoffAltitude` defined in the `TakeoffConfigSO`. It ascends at `TakeoffSpeed`. Once complete, it signals the `AiBrain` to clear the `RequestTakeoff` command.

4.7.1.10. BehaviorLanding

- **Goal:** Execute a controlled descent to a specific landing pad or target.
- **Logic:** Reads the `AiBrain.MoveTarget` as the landing destination. It navigates toward this point at the restricted `LandingSpeed` defined in the `LandingConfigSO`. Upon reaching the `ArrivalTolerance` distance, it idles and signals the `AiBrain` to clear the `RequestLanding` command.

4.7.1.11. BehaviorGuardian

- **Goal:** Escort and protect a designated allied ship, maintaining a specific formation offset.
- **Logic:** Reads the `AiBrain.MoveTarget` to identify the entity it is guarding. It calculates its desired position using the `FormationOffset` (defined in `GuardianConfigSO`) relative to the target's local space. It dynamically matches the target's current velocity to maintain a smooth formation.

4.8. Tactical Intelligence (SituationalAwarenessSystem and FactionManager)

The AI relies heavily on external systems for information about the world.

4.8.1. SituationalAwarenessSystem (SAS)

The `SituationalAwarenessSystem` (SAS) is a static utility class that acts as the intelligence broker. `AiBrain` and `AiConditionSO` submit `QueryRequest` structs to the SAS.

The SAS processes these requests, often by interfacing with the `FactionManager`, and returns a `QueryResponse`. This centralized system ensures that all AI agents use a consistent and optimized method for gathering tactical data.

Common `QueryType` examples include:

- `CheckDanger`: Determines if hostiles are within the ship's Engage/Disengage danger distances (hysteresis).
- `FindClosestHostile`/`FindClosestFriendlyLeader`/`FindNearestHomeBase`: Locates specific entities.

4.8.2. FactionManager and StarshipIdentity

The `FactionManager` is the central registry for all active ships, their allegiances, and roles. The SAS uses its optimized registries to fulfill queries.

The `StarshipIdentity` component on each ship provides the crucial context used by the SAS:

- `FactionID` and `RoleType`: Determines allegiance and strategic importance.
- **Tactical Distances:** Defines the ship's specific sensory ranges (Engage/Disengage Danger Distance) and operational limits (MaxHomeDistance).
- `PatrolWaypoints`: Stores scene-specific waypoints used by `BehaviorSentry`.

Chapter 5: Scene Setup & Game Modes

This chapter details how to configure your Unity scene to utilize the Starship Controller asset effectively. It is intended for users who have completed the "Getting Started" guide and are ready to build more complex scenarios, set up AI behaviors, and configure game modes like respawning.

We will focus on the scene-level configuration of essential managers and the setup of individual starship instances.

5.1 Essential Scene Managers

A functional scene requires several manager components to coordinate gameplay, physics, and AI. It is highly recommended to ensure these managers are present in any scene utilizing the core asset functionality.

5.1.1 EnvironmentManager

The `EnvironmentManager` is responsible for defining the global physical properties of the scene, primarily atmospheric density and drag coefficients.

Role:

- Provides a central source of environmental data used by the `AtmosphericStarshipController` for aerodynamic drag calculations.
- Can optionally override the scene's global gravity settings (`Physics.gravity`).

Setup:

1. Add the `EnvironmentManager` script to an empty GameObject in your scene (e.g., named "Managers").
2. It operates as a Singleton; ensure only one instance exists per scene.
3. Assign an `EnvironmentProfileSO` (Scriptable Object) to the `Current Profile` field. This profile defines the actual physics parameters (e.g., "Earth-like Atmosphere", "Low Gravity Moon", "Deep Space").

See the "Configuration Workflow & Profiles" chapter for details on creating and tuning `EnvironmentProfileSO` assets.

5.1.2 GameInitializer

The `GameInitializer` is a utility component that automates the setup process for the player's experience at the start of the scene.

Role:

- Locates the necessary components for the player: `PlayerShipDriver` (the player's ship) and `StarshipCameraController` (the main camera).
- Ensures the `InputProvider` exists, creating a fallback if necessary.
- Links the `PlayerShipDriver` with the `StarshipCameraController` by calling `InitializePlayerView`.
- Registers the `PlayerShipDriver` with the `GameStateManager` as the active player, which enforces the cursor lock required for flight control.

Setup:

1. Add the `GameInitializer` script to an empty GameObject in your scene (e.g., "Managers").
2. **Configuration (Optional):** You can manually assign the `Player Camera` and `Player Driver` in the Inspector if they are known ahead of time.
3. **Fallback (Recommended):** If left unassigned, the `GameInitializer` will automatically search the scene for `Camera.main` (and ensure it has a `StarshipCameraController`) and the first active `PlayerShipDriver` instance.

5.1.3 FactionManager

The `FactionManager` is the central nervous system for all AI operations and game mode logic. It maintains rosters of all active ships, categorized by faction and role, and manages the lifecycle of ships (including destruction and respawning).

Role:

- Registers all `StarshipIdentity` components in the scene.
- Provides optimized queries for the AI's `SituationalAwarenessSystem` (e.g., "Find closest hostile", "Find friendly leader").
- Manages the lifecycle of ships: tracking destruction events (`OnShipDestroyed`) and executing respawns if configured.
- Stores references to strategic locations (Bases, Waypoints, Respawn Points).

Setup:

1. Add the `FactionManager` script to an empty GameObject (e.g., "Managers").
2. It operates as a Singleton; ensure only one instance exists per scene.

Inspector Configuration:

The `FactionManager` requires configuration specific to the scene geography and game mode:

Respawn Configuration

These settings control where and when ships reappear after being destroyed.

- **Team A/B Respawn Point (Transform):** Assign an empty GameObject Transform for each team. This serves as the central location for respawning ships of that faction. If null, the FactionManager's own position is used as a fallback.
- **Respawn Radius (float):** The maximum distance (in meters) from the Respawn Point where a ship can reappear. This adds variance to the respawn location.
- **Respawn Time (float):** The delay (in seconds) between a ship's destruction and its reactivation at the respawn point.

Faction Rally Points (Home Bases)

These locations are used by certain AI behaviors (like Guardian or "Return to Base" logic) and for tactical queries (e.g., checking if a ship has strayed too far from home, using `StarshipIdentity.MaxHomeDistance`).

- **Team A/B Bases (List<Transform>):** Assign Transforms representing the home bases for each faction (e.g., space stations, carriers, or ground installations).

Faction Strategic Waypoints

These are general-purpose locations used by the `BehaviorTeamPatrol` or potentially custom game modes.

- **Team A/B Waypoints (List<Transform>):** Assign Transforms representing strategic points of interest for each faction.

Game Mode Objectives

- **Racecourse Center (Transform):** Used specifically by certain AI Actions or Conditions related to racing. Assign a Transform representing the approximate center of the race track.

Distance-Based Activation (Optimization) The FactionManager includes a highly efficient, built-in culling system. It runs a background coroutine to automatically deactivate distant AI ships and reactivate them when the player gets close, saving immense CPU overhead in large-scale battles.

- **Enable Distance Based Activation (bool):** Toggles the entire culling system on or off.
- **Deactivation Distance (float):** The distance (in meters) from the player at which eligible AI ships (those with `AllowDistanceBasedDeactivation` checked on their `StarshipIdentity`) will be completely deactivated.
- **Reactivation Distance (float):** The distance within which deactivated AI ships will be turned back on. *Crucial:* This should be slightly smaller than the Deactivation Distance (e.g., Deactivate at 5000m, Reactivate at 4800m) to create a hysteresis loop. This prevents ships from rapidly toggling on and off if they hover exactly at the boundary.
- **Distance Check Interval (float):** How often (in seconds) the manager evaluates ship distances. Higher values (e.g., 1.5s) spread out the CPU load, while lower values make the culling more responsive.

5.1.4 Game State Manager

The GameStateManager is a singleton that manages the overall application state, primarily focusing on pausing/resuming the game and enforcing cursor control. **Role:**

- Manages the current playing/paused state.
- Strictly enforces the cursor state (hidden and locked) when the player is actively flying, which is required for mouse-driven flight.

Setup:

- Add the `GameStateManager` script to an empty GameObject in your scene (e.g., "Managers").
- It operates as a Singleton; ensure only one instance exists per scene.

5.2 Configuring Individual Ships (StarshipIdentity)

Every ship in the scene, whether controlled by a player or AI, must have the `StarshipIdentity` component. This component defines the ship's allegiance, its role in the simulation, and its scene-specific behaviors.

5.2.1 Identity and Allegiance

- **Faction ID (Enum):** Assigns the ship to `TeamA` or `TeamB`. This determines who the ship considers hostile or friendly via the `FactionManager`.
- **Role (Enum):** Defines the ship's tactical role, influencing AI queries and behavior.
    - **Standard:** A typical combatant.
    - **Follower:** Often used for wingmen AI who prioritize following a Leader.
    - **Leader:** A ship designated as a tactical lead. The `FactionManager` optimizes queries for Leaders.
    - **Player:** Designates this ship as the player. The `FactionManager` caches this identity for quick access.

5.2.2 Scene Patrol Configuration (Sentry)

If you want an AI ship to patrol a specific route defined within the scene (using `BehaviorSentry`), you configure it directly on the `StarshipIdentity` instance. This allows different instances of the same prefab to patrol different routes.

- **Patrol Waypoints (List<Transform>):** Drag and drop empty GameObjects (Transforms) from the scene hierarchy into this list. This defines the specific route this ship instance will follow.
- **Patrol Pattern (Enum):**
    - **Loop:** The ship moves from the last waypoint back to the first, patrolling continuously.
    - **Sequential:** The ship moves through the waypoints once and then stops (enters Idle behavior if the Sentry action completes).
    - **Random:** The ship picks a random waypoint from the list each time it reaches its current destination.

5.2.3 Lifecycle Management

These settings determine how the ship behaves when destroyed, interacting with the `FactionManager`.

- **Can Respawn (bool):** If checked, the `FactionManager` will automatically handle the respawn process for this ship when it is destroyed, using the configured Respawn Time and Location.
- **Reset Weapons On Respawn (bool):** If checked, the ship's ammunition (managed by `ShipWeaponController`) will be fully restored when the ship respawns (specifically, when `HealthComponent.Initialize()` is called).

5.2.4 Tactical Configuration (Leash/Arrival)

These distances define the thresholds used by the AI's decision-making process and the `SituationalAwarenessSystem`.

- **Engage/Disengage Danger Distance:** Defines the hysteresis loop for threat detection. The AI considers itself "in danger" when a hostile enters the `Engage` distance, and only considers itself "safe" when all hostiles exceed the `Disengage` distance.
- **Max Home Distance:** The maximum distance the ship is allowed to stray from its nearest Home Base (defined in `FactionManager`).
- **Arrival Radius:** The distance used by 'Follower' AI (or behaviors like Gofer) to determine when they have reached their destination.

5.3 Example Scene Setups

5.3.1 Setting up Respawning

To create a standard deathmatch scenario with respawning:

1. Ensure a `FactionManager` exists in the scene.
2. Create empty GameObjects for respawn locations (e.g., "Respawn_TeamA", "Respawn_TeamB").
3. Assign these Transforms to the `Team A/B Respawn Point` fields on the `FactionManager`.

4. Set the desired `Respawn Time` (e.g., 5.0 seconds).
5. On each ship prefab or instance (including the player's ship), ensure the `StarshipIdentity` component is present.
6. Check the `Can Respawn` box on the `StarshipIdentity` for any ship that should reappear after destruction.

5.3.2 Setting up AI Patrols

There are two primary ways to set up patrols:

1. Scene-Specific Patrol (Sentry)

To create an AI that patrols a route unique to this specific instance:

1. Place the AI ship in the scene. Ensure it has `StarshipIdentity`, `AiBrain`, and `SimpleAiPilot` components.
2. Create the patrol route using empty GameObjects (e.g., "PatrolPoint1", "PatrolPoint2").
3. On the ship's `StarshipIdentity` component, assign these GameObjects to the `Patrol Waypoints` list and set the `Patrol Pattern` (e.g., `Loop`).
4. Configure the `AiBrain`: Ensure that the `Actions` list includes an `AiActionSO` that activates the `Sentry` behavior (`AiMode.Sentry`). The Sentry behavior automatically reads the waypoints configured on the `StarshipIdentity`.

2. Faction-Wide Patrol (TeamPatrol)

To create AI that patrol strategic points shared by the whole team:

1. Ensure a `FactionManager` exists.
2. Create strategic waypoints (e.g., "TeamA_Waypoint_Alpha").
3. Assign these to the `Team A/B Waypoints` list on the `FactionManager`.
4. Configure the `AiBrain` of the desired ships: Ensure their `Actions` list includes an `AiActionSO` that activates the `TeamPatrol` behavior (`AiMode.TeamPatrol`).

6. Configuration Workflow & Profiles (Scriptable Objects)

This chapter details the configuration system for the Starship Controller asset, which is built primarily around the use of **Scriptable Objects (SOs)**. Understanding this workflow is essential for customizing ship performance, weapon characteristics, AI behavior, environmental physics, and sensory parameters.

6.1. The Scriptable Object Workflow

The core philosophy of this asset is to decouple configuration data from the GameObjects and MonoBehaviours that use it. This is achieved using Scriptable Objects, which act as data containers living in your project assets folder, rather than being tied to a specific scene or prefab.

6.1.1. Benefits of Using Scriptable Objects

This approach offers several significant advantages:

- **Reusability:** A single configuration profile can be shared across multiple objects. For example, you can create one "Interceptor" `StarshipProfileSO` and assign it to 50 different enemy ships.
- **Easy Tweaking and Variants:** You can modify parameters in the profile asset, and the changes will automatically propagate to all objects using that profile, even during Play Mode. This makes balancing and creating variants much faster.
- **Decoupling Data:** Configuration data is kept separate from the functional logic of the components, leading to cleaner code and easier maintenance.
- **Cleaner Prefabs and Version Control:** Changes to configuration do not require modifications to Prefabs or scene files, leading to fewer merge conflicts and cleaner project history.

6.1.2. General Workflow

The workflow for using these configuration profiles involves three steps: Creating the asset, assigning the asset, and utilizing the data.

6.1.2.1. Creating a New Profile

To create a new configuration profile, navigate to the Unity menu bar or right-click in the Project window:

Assets > Create > [Relevant Category] > [Desired Profile Type]

The relevant categories are typically **Navigation System**, **Starship**, or **Starship AI**.

For example, to create a new ship profile: **Assets > Create > Navigation System > Starship Profile**

A new asset file will be created. It is recommended to name it descriptively (e.g., `SProfile_HeavyFreighter`, `WStats_GatlingGun`).

6.1.2.2. Assigning a Profile

Once created, the profile must be assigned to the corresponding component on your starship GameObject.

1. Select the starship GameObject in the Hierarchy or Scene view.
2. Locate the relevant component in the Inspector (e.g., `ShipProperties`, `ShipWeaponController`, `SimpleAiPilot`, `ShipSensorySystem`).
3. Drag the created profile asset from the Project window into the designated Scriptable Object field on the component (e.g., the `Profile SO` field on `ShipProperties`, or the `CurrentPersonality` field on `SimpleAiPilot`).

6.1.2.3. How Components Use Profiles

Many core components (like `ShipProperties` and `ShipSensorySystem`) feature a `useProfile` toggle in the Inspector.

- **If `useProfile` is enabled AND a profile is assigned:** The component will load its configuration parameters from the assigned Scriptable Object asset during initialization (e.g., in `Awake()` or

`OnEnable()`). The values visible in the Inspector on the component will update to reflect the loaded profile data.

- **If `useProfile` is disabled OR no profile is assigned:** The component will use the parameters manually set directly on the component instance in the Inspector. This is useful for quick testing or unique, one-off configurations.

It is highly recommended to use profiles for all production configurations to maintain consistency and organization.

6.2. Detailed Profile Breakdown

This section provides a comprehensive explanation of each configuration profile type.

6.2.1. StarshipProfileSO (Ship Physics and Handling)

Asset Menu: `Navigation System/Starship Profile` **Used By:** `ShipProperties` (which is utilized by `AtmosphericStarshipController`)

The `StarshipProfileSO` defines the fundamental physics characteristics, flight capabilities, and handling model of a starship. This profile dictates how heavy the ship feels, how fast it can go, and how agile it is.

6.2.1.1. Core Physics

- **mass** (float): The mass of the starship in kilograms (kg).
    - *Effect:* Higher mass requires more thrust to accelerate and more torque to rotate, making the ship feel heavier and less responsive.
- **maxSpeed** (float): The maximum velocity the ship can achieve under normal thrust, in meters per second (m/s). The physics engine clamps the velocity to this value (multiplied by the current boost multiplier).
- **rotationDamping** (float): Custom damping applied to the ship's angular velocity (rotation).
    - *Effect:* Higher values cause the ship to stop rotating more quickly when input is released, reducing drift and overshoot. This helps stabilize the ship.
- **angularDrag** (float): The standard Rigidbody angular drag. This works in conjunction with `rotationDamping`.
- **useGravity** (bool): Determines if the ship's Rigidbody is affected by the scene's global gravity settings.

6.2.1.2. Thrust Settings

These parameters define the force applied by the ship's engines.

- **forwardThrustPower** (float): The maximum force applied when thrusting forward (Local +Z axis), in Newtons (N).
- **reverseThrustPower** (float): The maximum force applied when thrusting backward or braking (Local -Z axis).
- **verticalThrustPower** (float): The maximum force applied when thrusting vertically (Local +Y axis), used for lift, hovering, and altitude correction.
- **thrustRampUpSpeed** (float): How quickly the engines respond to input changes.
    - *Effect:* Higher values make the throttle feel immediate; lower values simulate engine spool-up time.

6.2.1.3. Rotation Settings

These parameters define the torque applied for maneuvering.

- **pitchPower** (float): The maximum torque applied for pitching (Rotation around Local X axis).
- **yawPower** (float): The maximum torque applied for yawing (Rotation around Local Y axis).
- **rollPower** (float): The maximum torque applied for rolling (Rotation around Local Z axis).

6.2.1.4. Flight Model & Maneuverability

This section defines the advanced flight characteristics and limitations.

- **maneuverabilityCurve** (AnimationCurve): Defines how the ship's agility changes based on its current speed. This curve modulates the `pitchPower` and `yawPower`.
    - **X-Axis (Time):** Represents the ship's normalized speed (0.0 to 1.0, where 1.0 corresponds to `maxSpeed`).
    - **Y-Axis (Value):** Maneuverability multiplier (typically 0.0 to 1.0).
    - *Interpretation:* To create a realistic flight envelope, the curve often peaks at a mid-range speed (the "corner speed") and drops off at very low speeds (insufficient control authority) and very high speeds (excessive inertia/structural limits). A flat curve (value 1.0 across all speeds) provides consistent agility regardless of speed.
- **useGForceLimiter** (bool): Enables the G-force safety system (Flight Control System). If enabled, the ship's computer will automatically reduce thrust and torque inputs to prevent exceeding the `maxOverallGForce`.
- **maxOverallGForce** (float): The maximum G-force the ship can sustain before the limiter engages (e.g., 9-12G is typical for high-performance fighters).

6.2.1.5. Aerodynamics

- **aerodynamicDragModifier** (float): A multiplier applied to the environment's base drag values (defined in `EnvironmentProfileSO`).
    - *Effect:* A value less than 1.0 means the ship is more aerodynamic (less drag) than the standard model; a value greater than 1.0 means it experiences more drag.

6.2.1.6. Ground Effect

This simulates the increased lift (cushion of air) experienced when flying close to a surface.

- **groundEffectMaxAltitude** (float): The maximum altitude (in meters) above a surface where the ground effect is active.
- **groundEffectMaxForce** (float): The maximum upward force (in Newtons) applied when the ship is at zero altitude. The actual force is interpolated based on the `groundEffectCurve` defined in the `ShipProperties` component instance.

6.2.1.7. Input Settings

- **mouseSensitivity** (float): A multiplier for mouse input when controlled by the `PlayerShipDriver`.

6.2.1.8. Boost System

- **boostMultiplier** (float): The factor by which `forwardThrustPower` and `maxSpeed` are increased when boosting.
- **maxBoost** (float): The total capacity of the boost reservoir.
- **boostDrainRate** (float): The amount of boost consumed per second when active.
- **boostRechargeRate** (float): The amount of boost regenerated per second when inactive.
- **boostRechargeDelay** (float): The time (in seconds) after boosting stops before regeneration begins.
- **boostRampUpSpeed** (float): How quickly the boost effect engages and disengages.

6.2.1.9. LERP Mode (AI Optimization)

These settings define the simplified movement behavior when the `AtmosphericStarshipController` is switched to `StarshipMovementMode.Lerp` for performance optimization (typically for distant AI).

- **lerpCruiseSpeed** (float): The standard movement speed (m/s) used in LERP mode.
- **lerpCruisingAltitude** (float): The preferred altitude (m) the ship tries to maintain using simplified checks in LERP mode.
- **lerpVerticalSpeed** (float): The maximum vertical speed (m/s) used for altitude adjustments in LERP mode.
- **lerpRotationSpeed** (float): How quickly the ship rotates (degrees per second) based on AI input in LERP mode.
- **lerpGroundMask** (LayerMask): The physics layers used for simplified ground detection in LERP mode.

6.2.1.10. Example Use Cases

- **Interceptor:** Low mass, very high maxSpeed, high thrust, high rotation power, maneuverability curve optimized for high speed.
- **Heavy Freighter:** High mass, low maxSpeed, moderate thrust, low rotation power, high aerodynamicDragModifier.

6.2.2. WeaponStats (Weapon Characteristics)

Asset Menu: `Starship/Weapon Stats` **Used By:** `ShipWeaponController`, `ProjectileBase` (and derived classes)

The `WeaponStats` profile defines the characteristics of a specific weapon system, including its type, firing rate, ammunition capacity, projectile behavior, and guidance capabilities.

6.2.2.1. General

- **weaponName** (string): A descriptive name for the weapon (e.g., "Mk II Laser Cannon").
- **type** (WeaponType): The category of the weapon (`Laser`, `Rocket`, `Bomb`). This influences how the `ShipWeaponController` handles accuracy models and guidance logic.
- **projectilePrefab** (GameObject): The prefab instantiated when the weapon is fired. This prefab must contain a component derived from `ProjectileBase` (e.g., `LaserProjectile`, `GuidedRocketProjectile`).

6.2.2.2. Firing Mechanics

- **refireRate** (float): The cooldown time (in seconds) between shots.
    - *Effect:* Lower values result in a higher rate of fire (e.g., 0.1s is 10 shots per second).
- **maxAmmo** (int): The maximum ammunition capacity. Set to 0 for infinite ammo (typical for energy weapons).

6.2.2.3. Laser Specific (Accuracy)

These settings apply primarily when `type` is `Laser`.

- **maxRange** (float): The maximum effective range of the weapon (in meters).
- **accuracyFalloffStart** (float, 0.0-1.0): The distance (as a fraction of `maxRange`) at which the weapon's accuracy starts to decrease.
    - *Example:* If `maxRange` is 1000m and this is 0.8, accuracy is perfect up to 800m.
- **minAccuracy** (float, 0.0-1.0): The lowest possible accuracy at `maxRange`. Accuracy is interpolated between 1.0 (at `accuracyFalloffStart`) and this value (at `maxRange`). Lower values result in greater spread at distance.

6.2.2.4. Rocket/Projectile Specific

- **projectileSpeed** (float): The speed of the projectile (m/s). This value is crucial for the AI (`AiWeaponController` and `InterceptionCalculator`) to calculate interception points. It should match the actual speed of the projectile prefab.

6.2.2.5. Guided Weapon Settings

These settings apply when `isGuided` is true (typically only for `WeaponType.Rocket`).

- **isGuided** (bool): If true, the weapon uses guidance logic. When fired by the player or AI, if the driver is actively engaged (see `ShipInputState.IsTargetEngaged`), the missile will track the `ShipWeaponController`'s `CurrentTarget`. If not engaged, it fires straight (unguided).
- **maxLockAngle** (float): The maximum angle (in degrees) off the ship's forward axis where a lock can be maintained and the weapon fired. Defines the engagement cone for guided weapons.
- **maxLockDistance** (float): The maximum distance (in meters) for maintaining a lock and firing the guided weapon.

6.2.2.6. Example Use Cases

- **Rapid Fire Laser:** Type Laser, low refireRate (0.1s), maxAmmo 0, high accuracy, high projectileSpeed.
- **Dumbfire Rockets:** Type Rocket, isGuided false, moderate refireRate, limited maxAmmo, moderate projectileSpeed.
- **Guided Missile:** Type Rocket, isGuided true, high refireRate (2.0s), low maxAmmo, specific lock parameters.

6.2.3. FlightPersonalitySO (AI Flight Style)

Asset Menu: `Starship AI/Simple/Flight Personality` **Used By:** `SimpleAiPilot`

The `FlightPersonalitySO` defines *how* an AI pilot flies the ship. It controls the AI's rotation logic, aggression, precision, and safety protocols, allowing you to create distinct pilot skill levels and behaviors. It provides the tuning parameters for the `SimpleAiPilot` component.

6.2.3.1. Core Style

- **Style** (FlightStyle): The fundamental algorithm used for rotation control.
    - `Calculated`: Uses PID (Proportional-Integral-Derivative) controllers for smooth, precise rotation control. Generally the most stable and accurate style, but requires tuning the PID parameters.
    - `Focused`: Uses simpler proportional control. Prioritizes aligning with the target before committing significant thrust (see `FocusedThrustThresholdAngle`).
    - `Reckless`: Uses simple proportional control. Applies full power constantly, "turning and burning" simultaneously. Aggressive but often overshoots and can be erratic.

6.2.3.2. General Movement Parameters

- **AlignmentConeAngle** (float, degrees): The "deadzone" angle. If the target direction is within this cone relative to the ship's forward vector, the AI considers itself aligned and stops applying rotation inputs.
    - *Effect:* Larger values prevent "wobble" or oscillation around the target but reduce precision. Typical range: 1.0 to 5.0.
- **SpeedMultiplier** (float): A multiplier applied to the ship's base `maxSpeed` (from `StarshipProfileSO`).
    - *Effect:* Allows AI pilots to fly faster (e.g., >1.0 for aces) or slower (e.g., <1.0 for rookies) than the ship's nominal maximum.
- **AllowBoost** (bool): Whether this personality permits the AI to use the boost system when trying to close distance quickly.

6.2.3.3. Focused/Reckless (Proportional Control) Parameters

These parameters apply when the `Style` is `Focused` or `Reckless`.

- **RotationAggressiveness** (float): The main gain factor for proportional control.
    - *Effect:* Higher values result in snappier, more aggressive turns, but increase the risk of overshoot.
- **FocusedThrustThresholdAngle** (float, degrees): In `Focused` mode only, the angle at which the AI transitions from minimal thrust (while turning) to significant thrust (when aligned).

6.2.3.4. Calculated (PID Control) Parameters

These parameters apply when the `Style` is `Calculated`. PID tuning is highly dependent on the ship's mass and torque (defined in `StarshipProfileSO`).

- **PitchKp, YawKp** (float): Proportional Gain (P). Controls the immediate reaction strength relative to the error (misalignment).
    - *Effect:* Higher Kp results in faster reaction time. If too high, it causes oscillation and overshoot.
- **PitchKi, YawKi** (float): Integral Gain (I). Corrects accumulated error over time (e.g., if the ship consistently under-turns).
    - *Effect:* Often kept low or zero for flight control to prevent "integral windup".

- **PitchKd, YawKd** (float): Derivative Gain (D). Dampens the response based on the rate of change of the error.
    - *Effect:* Higher Kd reduces overshoot and oscillation, smoothing the approach to the target direction. Crucial for stability.
- **IntegralClamp** (float): The maximum value the Integral term can accumulate. This prevents excessive windup which can cause the AI to circle the target. A value of 1.0 is usually sufficient.

6.2.3.5. Terrain Avoidance

- **UseMinimumAltitude** (bool): If true, the `SimpleAiPilot` will engage vertical thrusters (as an override to the current behavior) to maintain a minimum height above the ground detected by the `ShipSensorySystem`.
- **MinimumAltitude** (float): The desired minimum distance (in meters) to keep from the ground.

6.2.3.6. Upright Stabilization (Cosmetic)

- **MaintainUprightOrientation** (bool): If enabled, the AI will gently apply roll inputs to align its local up vector with the world up vector (Vector3.up).
- **RollKp, RollKi, RollKd** (float): PID parameters specifically for the upright stabilization roll control (used only in `Calculated` style if `MaintainUprightOrientation` is true).

6.2.3.7. Example Use Cases

- **Rookie Pilot:** Style Focused, lower RotationAggressiveness, larger AlignmentConeAngle, AllowBoost false.
- **Ace Pilot:** Style Calculated, well-tuned PID values (higher Kp and Kd), small AlignmentConeAngle, AllowBoost true.
- **Transport Pilot:** Style Calculated, emphasis on stability (higher Kd), strict MinimumAltitude enforcement.

6.2.4. SensorySettingsSO (AI Perception and Avoidance)

Asset Menu: `Navigation System/Sensory Settings` **Used By:** `ShipSensorySystem`

The `SensorySettingsSO` defines the parameters for the ship's sensory system, primarily used by the AI for ground detection and forward obstacle avoidance.

6.2.4.1. Ground Detection

- **maxGroundCheckDistance** (float): The maximum distance (in meters) the system checks downwards for terrain. This determines the maximum readable altitude.

6.2.4.2. Obstacle Detection (Passive Scan)

This defines the passive forward scan that runs periodically.

- **forwardCheckDistance** (float): How far ahead (in meters) the system scans for obstacles along the ship's velocity vector. This is the distance at which an avoidance maneuver is triggered.

- **checkRadius** (float): The radius (in meters) of the sphere cast used for the forward check, approximating the ship's collision volume.

6.2.4.3. Avoidance Scanning (Active Scan)

When an obstacle is detected by the passive scan, the system performs an active, multi-ray scan to find an escape route.

- **coneAngle** (float, degrees): The angle of the cone used for the active avoidance scan. A wider cone covers more area.
- **coneRayCount** (int): The number of rays cast within the cone to search for clear paths.
    - *Effect:* Higher counts provide better resolution for finding gaps but increase the performance cost of the active scan.

6.2.4.4. Avoidance Behavior

- **avoidanceDuration** (float, seconds): How long the `ShipSensorySystem` remains in the "Avoiding" state, continuously providing the avoidance vector to the AI pilot.
- **cooldownDuration** (float, seconds): The time after an avoidance maneuver completes before the system returns to the "Searching" state (resuming passive scanning).

6.2.4.5. Decision Bias

These parameters influence which clear path the system prefers during an active scan if multiple options exist.

- **upwardBiasBonus** (float): A bonus score added to potential escape paths that point upwards (generally safer).
- **turningInertiaBiasBonus** (float): A bonus score added to paths that align with the ship's current angular velocity (momentum), favoring the continuation of the current turn direction.

6.2.5. EnvironmentProfileSO (Aerodynamics and Drag)

Asset Menu: (Inferred) `Navigation System/Environment Profile` **Used By:** `EnvironmentManager` (Inferred Static Access), utilized by `AtmosphericStarshipController`

The `EnvironmentProfileSO` defines the aerodynamic properties of the atmosphere, specifically the air density and drag characteristics. This allows you to simulate different environments (e.g., dense atmosphere vs. near-vacuum).

The drag model uses both linear and quadratic components, applied differentially to the ship's forward and sideways movement. The values in this profile are the *base* environmental values, which are then multiplied by the ship's `aerodynamicDragModifier` (from `StarshipProfileSO`).

6.2.5.1. Forward Drag

These parameters define the drag experienced when moving along the ship's forward axis (+Z).

- **baseForwardLinearDrag** (float): The coefficient for linear drag (proportional to velocity).

    - *Effect:* Dominant at lower speeds. Higher values increase "stickiness" and reduce coasting distance.
- **baseForwardQuadraticDrag** (float): The coefficient for quadratic drag (proportional to the square of velocity).
    - *Effect:* Dominant at higher speeds. Higher values significantly increase the force required to maintain high velocity and effectively determine the terminal velocity in the atmosphere.

6.2.5.2. Sideways Drag

These parameters define the drag experienced when moving perpendicular to the ship's forward axis (e.g., drifting sideways or moving vertically).

- **baseSidewaysLinearDrag** (float): The coefficient for linear drag on sideways movement.
- **baseSidewaysQuadraticDrag** (float): The coefficient for quadratic drag on sideways movement.
    - *Note:* Sideways drag is typically much higher than forward drag for aerodynamic shapes, making sustained sideways flight inefficient and quickly damping out drift.

6.2.5.3. Example Use Cases

- **Earth-like Atmosphere:** Moderate values for all drag components.
- **Space/Vacuum:** All drag components set to 0.
- **Dense Atmosphere/Liquid:** Very high values for all drag components.

6.3. Organization Tips

As your project grows, maintaining a clear folder structure for your configuration assets is essential. We recommend the following organization within your `Assets` folder:

Assets/

└── Configurations/

  ├── AI/

  │ ├── FlightPersonalities/ (FlightPersonalitySO)

  │ │ ├── P_Rookie.asset

  │ │ └── P_Veteran.asset

  │ ├── BehaviorConfigs/ (e.g., AttackConfigSO, SentryConfigSO)

  │ ├── Actions/ (AiActionSO)

  │ └── Conditions/ (AiConditionSO)

  ├── Environments/

  │ └── Env_StandardAtmosphere.asset (EnvironmentProfileSO)

  ├── Ships/

```
│   ├── Profiles/
│   │   ├── SP_Fighter.asset
│   │   └── SP_Freighter.asset (StarshipProfileSO)
│   └── Sensory/
│       └── SS_StandardSensors.asset (SensorySettingsSO)
└── Weapons/
    └── WeaponStats/
        ├── WS_Laser_Light.asset
        └── WS_Missile_Guided.asset (WeaponStats)
```

Chapter 7: Stationary Turret Systems

The Veridian Starship Controller asset isn't just for flying vehicles; it also includes robust support for stationary defensive turrets. The codebase provides two distinct methods for creating turrets, allowing you to balance simplicity and tactical depth.

7.1 The Lightweight Turret (`StationaryTurretAI.cs`)

This is a standalone, all-in-one script perfect for simple "shoot-on-sight" base defenses. It bypasses the complex `AiBrain` and handles its own target detection, line-of-sight checks, rotation, and predictive aiming using the `InterceptionHelper`.

Setup:

1. Create your turret GameObject and ensure it has a `ShipWeaponController` component configured with `WeaponStats` and fire points.
2. Attach the `StationaryTurretAI` script to the root of the turret.

Configuration:

- **Targeting Configuration:**
  - `TargetLayerMask`: The physics layers the turret considers as valid targets.
  - `DetectionRange`: The maximum distance (in meters) the turret will search for targets.
  - `ObstructionLayerMask`: Layers (like terrain or buildings) that block the turret's line of sight.
- **Rotation:**
  - `RotationPivot`: The specific child Transform of the turret that should rotate to face the target (e.g., the gun barrel or turret head). If left null, the entire root object will rotate.
  - `RotationSpeed`: How fast the pivot turns (degrees per second).

    - `FiringAngleThreshold`: The turret will only pull the trigger if its forward vector is within this many degrees of the calculated interception point.

7.2 The Advanced AI Turret (`TurretRotator.cs`)

If you want a turret that integrates fully with the `FactionManager`, obeys the `SituationalAwarenessSystem`, and uses the prioritized `AiBrain` to make decisions (e.g., prioritizing leaders over followers, or retreating its guns when damaged), you use the `TurretRotator`.

In this setup, the `TurretRotator` effectively replaces the `SimpleAiPilot`. The `AiBrain` thinks, the `AiWeaponController` calculates the firing solution, and the `TurretRotator` physically moves the gun.

Setup:

1. Create your turret GameObject.
2. Add the core identity and combat scripts: `StarshipIdentity` (Set Role to *Turret*), `HealthComponent`, and `ShipWeaponController`.
3. Add the AI scripts: `AiBrain` and `AiWeaponController`. (You do not need an `AtmosphericStarshipController` or `SimpleAiPilot`).
4. Attach the `TurretRotator` script.

Configuration (`TurretRotator`):

- **RotatingPart:** The Transform representing the moving part of the turret (head/arm). This part handles both Yaw and Pitch and *must* contain the weapon fire points defined in the `ShipWeaponController`.
- **RotationSpeed:** The maximum rotation speed in degrees per second.
- **ResetToInitialRotationWhenIdle:** If true, the turret will smoothly return to its default resting orientation when it loses its target or the `AiBrain` enters an idle state.

(Note: For Advanced AI Turrets, ensure the `AimingTransformOverride` field on the `AiWeaponController` is set to the same Transform as your `RotatingPart`. This ensures the AI calculates its firing cone from the moving barrel, not the static base!)